



Towards Optimised Use of Software and Statistical Infrastructure for Single-Cell Proteomics

Guyot Léopold

August 13, 2025

Thèse de Master soumise en vue de satisfaire aux conditions d'obtention du diplôme de Master en Bioinformatique et Modélisation

Supervisors:

Gatto Laurent
Computational Biology and Bioinformatics lab (CBIO)
de Duve Institute
Université Catholique de Louvain (UCL), Belgium

Defrance Matthieu
Machine Learning Group And Interuniversity Institute of Bioinformatics in Brussels
Computer Science Department
Université libre de Bruxelles (ULB), Belgium



Abstract

This master thesis addresses two main challenges in the analysis of mass spectrometry-based single-cell proteomics (SCP) data: the scalability of computational infrastructure and the evaluation of modelling strategies suited to the unique characteristics of SCP datasets.

First, we conducted an in-depth performance analysis of the existing *QFeatures* and *scp* packages, that are core components of the R/Bioconductor ecosystem for SCP data processing. Through detailed profiling of runtime and memory usage, we identified key bottlenecks. Guided by this analysis, we implemented targeted optimisations, including algorithmic improvements, change of internal data structures and better guidelines to conduct analyses. These changes resulted in a 96% reduction in execution time and approximately 70% reduction in memory consumption, significantly improving the interactivity and scalability of the framework. The optimised version is already available in the Bioconductor development branch and will be included in the next stable release (3.22).

Second, we explored the use of pseudobulking, an approach widely adopted in single-cell transcriptomic, as a scalable modelling strategy for MS-SCP data. Using a simulation-based framework with inserted artificial effects, we benchmarked pseudobulk methods (mean, median, sum, robust summary) against state-of-the-art tools such as *msqrob2* and *scplainer*. We found that mean and median aggregation yield competitive performance with substantially lower computational costs. On the other hand, other aggregation strategies (e.g., sum, robustSummary) were either less accurate or more computationally expensive.

We also discussed the limitations of our simulation framework and emphasised the need for more multi-patients multi-conditions datasets.

Together, this work delivers concrete software improvements and a forward-looking evaluation of scalable statistical strategies. It contributes to the robustness, usability, and future readiness of computational tools in the rapidly evolving field of single-cell proteomics.

Abbreviations

Abbreviation	Meaning
DDA	Data Dependent Acquisition
DIA	Data Independent Acquisition
FACS	Fluorescence-Activated Cell Sorting
LC	Liquid Chromatography
LCM	Laser Capture Microdissection
MS	Mass Spectrometry
MS-SCP	Mass Spectrometry-based Single-Cell Proteomics
RAM	Random Access Memory
SCE	SingleCellExperiment
SCP	Single-Cell Proteomics
scRNAseq	Single-Cell transcriptomics
SE	SummarizedExperiment

List of Figures

1	Scheme of a simplified SCP experimental workflow.	5
2	Scheme of the <i>SummarizedExperiment</i> class.	8
3	Scheme of the <i>QFeatures</i> infrastructure in the context of SCP data.	9
4	Overview of the main operation types performed on a <i>QFeatures</i> object.	11
5	<i>QFeatures</i> scheme of Leduc et al. 2022 analysis.	18
6	Scheme of the simulation framework.	22
7	Real pipeline benchmark global results.	25
8	Real pipeline benchmark per step results.	26
9	Individual components benchmark global results	27
10	Individual components benchmark <i>aggregateFeaturesOverAssays</i> , <i>subsetByColData</i> and <i>joinAssays</i> functions results	28
11	Impact of the different parameters on the size of the <i>QFeatures</i> object.	30
12	Challenging dataset benchmark results.	31
13	<i>subsetByColData</i> optimisation results.	32
14	<i>aggregateFeatures</i> optimisation results.	33
15	<i>SummarizedExperiment</i> vs <i>SingleCellExperiment</i> performances.	34
16	Simulation Quality Control.	36
17	Per shift models performances.	37
18	Per number of cells models performances.	38
19	Models runtime comparison.	39
20	Implemented optimisations results summary.	41

List of Tables

1	Comparative table of some MS methods.	5
2	<i>QFeatures</i> object memory usage benchmark parameter values.	20
S1	Raw results of the models performance at different shift values.	54
S2	Raw results of the models performance at different number of cells per condition.	55

Contents

Abstract	ii
Abbreviations	iii
List of Figures	iv
List of Tables	iv
Contents	v
Introduction	1
1. Background	1
1.1. Single Cell Proteomics	1
1.2. Software Infrastructure	6
1.3. Current challenges in SCP	13
2. Research questions	14
3. Methodology and Thesis Outlines	15
Materials and Methods	16
1. Datasets	16
1.1. Leduc et al. 2022	16
1.2. Brunner et al. 2022	16
1.3. Bulk HeLa control samples (Webel et al. 2024)	16
2. Benchmarks	16
2.1. Real Pipeline Benchmark	17
2.2. Individual Components Benchmark	19
2.3. QFeatures Object Memory Usage Benchmark	20
2.4. Challenging Dataset Benchmark	20
2.5. VerR	20
3. PseudoBulking	21
3.1. Simulated Data Generation	21
3.2. Data Aggregation and Modelling	23
Results	24
1. Benchmarks	24
1.1. Real Pipeline Benchmark	24
1.2. Individual Components Benchmarks	26
1.3. QFeatures Object Memory Usage Benchmark	28
1.4. Challenging Dataset Benchmark	29
2. Optimisations	31
2.1. subsetByColData Optimisation	31
2.2. Aggregation Optimisation	32
2.3. Using <i>SummarizedExperiment</i>	33
3. PseudoBulking	35
3.1 Simulation Quality Control	35
3.2 Impact of the shift value on performances	36
3.3 Impact of the number of cells on performances	37
3.4 Runtime analysis of the modelling methods	38
Discussion	40

1. Optimisations	40
1.1 Implemented Optimisations	40
1.2 Looking Ahead: Potential Optimisations	41
1.3 Beyond Performance: Knowing When to Stop	42
2. Pseudobulking	43
2.1 Insights from Simulation Studies	43
2.2 Simulation Limitations	44
2.3 Towards More Realistic Simulations	45
Conclusions	47
Acknowledgments	49
Bibliography	50
Annexes	54
Annex 1:	54
Annex 2:	55

Introduction

1. Background

1.1. Single Cell Proteomics

Defining Single-Cell Proteomics

Single-cell proteomics (SCP) refers to experimental and computational strategies aimed at identifying and quantifying proteins at the level of individual cells. Unlike bulk proteomics, which averages measurements across thousands or millions of cells, SCP resolves the heterogeneity between individual cells within a tissue. Characterizing this cell-to-cell variability is crucial in contexts such as cancer research, where rare resistant subpopulations may evade treatment [1, 2].

SCP is conceptually similar to single-cell transcriptomics (scRNA-seq), in that both approaches provide insight into cellular heterogeneity. However, while scRNA-seq quantifies mRNA levels as a proxy for gene expression, it does not fully capture the functional state of the cell [3]. Proteins are the primary effectors of cellular function. Their abundance, localization, and post-translational modifications (PTMs) are essential determinants of a cell's biological activity. For example, changes in protein abundance, PTMs, degradation, or secretion may not be reflected at the transcript level [4]. These aspects are especially important in contexts such as cancer, immune cell activation, or cellular differentiation, where post-transcriptional regulation plays a major role [5, 6, 7].

Several experimental approaches have been developed for SCP, each with distinct advantages and limitations. These can be broadly grouped into antibody-based methods (including cytometry and microscopy) and mass spectrometry-based SCP (MS-SCP) [8].

Antibody-based SCP: Cytometry and Microscopy Antibody-based SCP uses affinity reagents to detect specific target proteins in single cells. Cells or tissue sections are incubated with labeled antibodies that bind to the proteins of interest. These labels can include fluorophores (flow cytometry or fluorescence microscopy), heavy metal isotopes (mass cytometry or CyTOF [9]), or DNA barcodes (CITE-seq [10]). In the case of microscopy-based proteomics, labeled antibodies are visualized using optical or fluorescence microscopy, enabling spatially resolved detection of proteins in situ. Multiplexed imaging strategies, such as cyclic immunofluorescence [11] and CODEX [12], allow iterative staining and imaging to measure dozens of proteins while retaining subcellular localisation information.

Antibody-based methods offer relatively low per-cell costs and high throughput, particularly for flow cytometry, which can measure thousands of cells per second. Mass cytometry enables measurement of approximately 40–60 proteins per cell. Microscopy-based approaches, while generally lower in throughput, provide the unique advantage of preserving spatial context and allowing the analysis of subcellular protein distributions. Moreover, antibody-based approaches can be used to study specific post-translational modifications (PTMs) by employing PTM-specific antibodies, enabling PTM-SCP analyses [5].

However, these approaches are inherently targeted: only pre-selected proteins with validated antibodies can be measured, limiting the ability to discover new proteins or capture system-wide expression changes. The number of proteins measured per cell is typically in the range of 10–60. Sensitivity and specificity also depend on the quality of the antibodies used.

Mass spectrometry-based Single-Cell Proteomics (MS-SCP) MS-SCP uses highly sensitive mass spectrometry workflows to detect and quantify proteins in individual cells without the need for affinity reagents. This approach is inherently untargeted, enabling unbiased, proteome-wide anal-

ysis and the discovery of novel protein markers. MS-SCP can detect thousands of proteins per cell under optimized conditions. In addition, specialized workflows enable spatially resolved single-cell proteomics [13].

Despite these advantages, MS-SCP faces significant technical challenges. It has lower throughput than antibody-based methods and requires highly sensitive and efficient sample preparation due to the impossibility of protein amplification. Indeed, a key difference between transcriptomics and proteomics approaches is that RNA can be amplified (via PCR [14]), while proteins cannot. As a result, MS-SCP experiments are more expensive and technically demanding. Current datasets are often sparse, with a high proportion of missing values, necessitating robust experimental designs and advanced computational methods to manage noise and sparsity.

This work will focus on the mass-spectrometry approach, specifically on the quantitative analysis associated with it. Note that any further use of the SCP acronym will refer to Mass spectrometry-based Single-Cell Proteomics.

Application of MS-based Single-Cell Proteomics

Single-cell proteomics has been successfully applied to answer diverse biological questions that were previously inaccessible using bulk or transcriptomic approaches. These applications span a wide range of fields, including cancer biology, immunology, and developmental biology.

For instance, Brunner et al. [15] used mass spectrometry-based SCP to investigate how HeLa cells respond to cell cycle arresting drugs at the proteomic level. By quantifying proteins in over 400 individual cells, they could resolve distinct cell cycle phases, identify key regulators, and reveal novel markers of G2/M transition. Their study also highlighted how protein-level measurements provide more stable and deterministic insights compared to the transcriptomic data.

Similarly, Leduc et al. [16] applied SCP to dissect heterogeneity in melanoma cells. They discovered a subpopulation of cells in a drug resistant primed state, characterized by unique metabolic features and a G1-like proteomic signature. This illustrates how SCP can reveal functional subpopulations within a seemingly homogeneous cell line, this insights are critical for understanding treatment resistance.

Beyond cancer, SCP has been used to study immune cell activation, where dynamic changes in signalling proteins and post-translational modifications occur rapidly and are not captured at the transcript level [17]. In developmental biology, SCP has enabled the profiling of early embryonic cells, helping to elucidate lineage specification based on protein expression patterns [3].

Altogether, these examples show the utility of SCP to discover subtle yet biologically significant variations in protein abundance and function that cannot be observed with scRNAseq or bulk proteomics, making it a suitable tool for systems-level understanding of cellular behaviour.

Mass Spectrometry-Based Single-Cell Proteomics Methods

Mass spectrometry-based single-cell proteomics is a complex experimental workflow, that originate from MS-based bulk proteomics. This section will be a short overview of the methods used in MS-SCP. If the reader is interested, several well documented reviews exist [18, 19, 20, 21, 22]. Below, we outline the key stages of the SCP workflow and describe the major methodological variants in use today. These stages are illustrated in the Figure 1.

Sample Preparation The first step of the SCP workflow is the isolation of individual cells. Several manual or automated methods are available. Capillary aspiration is a approach where a fine glass capillary is used to manually collect individual cells under a microscope. While this method offers precise control, it is inherently low-throughput and labour intensive. Laser Capture Microdissection (LCM) is another manual technique in which a focused laser beam is used to excise and collect

specific cells or regions from a tissue section [23]. LCM is particularly useful when spatial context is important, although it also suffers from limited throughput. On the automated side, Fluorescence-Activated Cell Sorting (FACS) [24] is widely used in single-cell proteomics. It enables high-throughput sorting of cells based on fluorescent markers and physical properties such as size and granularity. Another advanced automated method is imaging-based cell sorting. This approach allows for the automated selection of cells based on morphology, fluorescence, or spatial context. One popular instrument used for imaging-based cell sorting is the cellenOne [16].

Then the cells are lysed using MS-compatible lysis buffers or mechanical lysis method (ultrasonication or freeze-thaw cycle). The extracted proteins are digested into peptides by the use of Trypsin and/or Lys-C.

This sample preparation step is performed on specialised reaction platforms designed to minimize sample loss and enable efficient processing of really small quantities of material. These platforms can be categorized into four types: capillary probes, multi-well plates, microfabricated chips, and glass slides. Capillary probe-based systems rely on fine glass or metal capillaries to aspirate, lyse, and digest single cells in minimal volumes, offering high sensitivity and control, though with limited throughput. Multi-well plates, typically in 96 or 384 well formats, are commonly used due to their compatibility with automated liquid handling systems and scalability, though they present challenges with reagent evaporation and surface adsorption. Microfabricated chips, such as nanowell or droplet-based platforms, use microfluidic technologies to confine reactions in nanoliter volumes, significantly reducing sample loss and improving reproducibility. Finally, glass slide-based systems like nanoPOTS employ hydrophilic-hydrophobic patterning to confine reactions in discrete spots, enabling ultra low volume processing [25]. Each platform represents a trade-off between throughput, sensitivity, and technical complexity, and the choice of platform often depends on the specific goals and constraints of the SCP experiment.

Sample labelling To increase throughput in MS-based single-cell proteomics, multiplexing techniques can be employed. This is typically achieved by chemically labelling peptides using either isobaric or non-isobaric tags, allowing simultaneous analysis of several cells in one run.

Isobaric labelling methods, such as Tandem Mass Tags (TMT), are the most established approach. TMT enables high-throughput quantification by labelling peptides from different cells with chemically identical tags that fragment to produce distinct reporter ions in MS/MS. This allows simultaneous analysis of up to 16 samples in one experiment. However, TMT faces challenges such as co-isolation interference and increased background noise, which reduce quantitative accuracy and proteome coverage [3]. To mitigate these limitations, the carrier proteome concept is often used in single-cell TMT workflows: a bulk carrier sample (typically 20–200 times the single-cell amount) is added to boost peptide signals, improving identification rates while still enabling quantification of the single-cell channels.

Other multiplexing strategies include timePlex [26] and plexDIA [27]. timePlex extends the multiplexing concept by encoding samples with retention time shifts, enabling multiplexed analysis with reduced spectral interference and improved quantitative accuracy. plexDIA is a data-independent acquisition (DIA) approach designed for multiplexed experiments, combining non-isobaric mass tags labelling with DIA methods to improve sensitivity and precision while maintaining high throughput.

Label-free (LF) quantification offers an alternative to labelling by analysing single cells individually without chemical tags. LF workflows are simpler, avoid ratio compression and labelling biases, and typically provide improved sensitivity and proteome coverage per cell. However, LF approaches suffer from lower throughput since each cell requires a separate run, increasing instrument time and cost.

The choice of labelling strategy depends on the experimental goals: multiplexing methods (TMT, timePlex, plexDIA) are advantageous for high-throughput studies, while label-free approaches remain attractive for maximising proteome depth and quantitative accuracy per cell, at the cost of reduced sample throughput.

Liquid chromatography Prior to mass spectrometry analysis, peptide mixtures are typically separated by liquid chromatography (LC). LC plays a critical role in reducing sample complexity and increasing the dynamic range of detectable peptides. In SCP workflows, nano-flow LC systems are often used to maximize sensitivity, as they allow for more efficient ionization and improved peptide recovery [28]. Peptides are loaded onto a chromatographic column and eluted over time using a gradient of solvents. This temporal separation ensures that peptides enter the mass spectrometer at different time points, minimizing ion suppression and enabling more accurate quantification and identification. The efficiency of peptide separation in LC directly impacts the depth and reproducibility of proteome coverage in single-cell experiments.

Mass spectrometry data acquisition Mass spectrometry (MS) is an analytical technique that measures the mass-to-charge ratio (m/z) of ionized molecules to identify and quantify peptides and proteins. In MS-based proteomics, peptides are first ionized and introduced into the mass spectrometer, where their m/z values are measured. Selected ions are then fragmented to produce MS/MS spectra, which are used to infer peptide sequences and their abundances.

In mass spectrometry-based single-cell proteomics, two main acquisition strategies are commonly used: data-dependent acquisition (DDA) and data-independent acquisition (DIA). In DDA, the mass spectrometer first performs an MS1 scan to detect precursor ions and then selects the top-N most intense ions for fragmentation in sequential MS2 scans. This approach enables high-quality identification of abundant peptides but is biased toward high-abundance species, often resulting in missing data for low-abundance peptides across samples. In contrast, DIA divides the full m/z range into predefined windows and fragments all ions within each window simultaneously. This strategy captures MS2 information for all detectable ions regardless of abundance, thereby increasing data completeness and reproducibility. However, DIA produces more complex spectra that require advanced computational deconvolution for accurate peptide identification and quantification.

Technological and historical evolution of SCP

Mass spectrometry-based single-cell proteomics has improved very quickly since its beginning with remarkable progress in sample preparation, mass spectrometry devices, and data acquisition strategies (Table 1).

One of the earliest uses of SCP was the Oil-Air-Droplet (OAD) platform by Li et al. in 2018 that employed a label-free, DDA-based approach and successfully quantified approximately 50 proteins per cell from one hundred cells [29]. The same year, Zhu et al. developed nanoPOTS (Nanodroplet Processing in One Pot for Trace Samples) that greatly minimized the loss of samples through the reduction of reaction volumes, therefore improving the sensitivity and enabling quantification of about 330 proteins per cell from 140 cells using label-free DDA [25].

A crucial conceptual change occurred with the development of SCoPE-MS by Budnik et al. in 2018. This method was the first to use isobaric labeling (TMT-6plex) and a carrier proteome to increase peptide signal, allowing the identification of over 1,500 proteins from 190 single cells [3]. Building on this new method, Tsai et al. integrated isobaric labelling into the nanoPOTS protocol to create the nanoPOTS-iBASIL method, which used multiplexing (TMT-13plex) and had a great proteome coverage thanks to low sample losses [30].

Following optimisation was made by SCoPE2, designed in 2021, which improved on the fun-

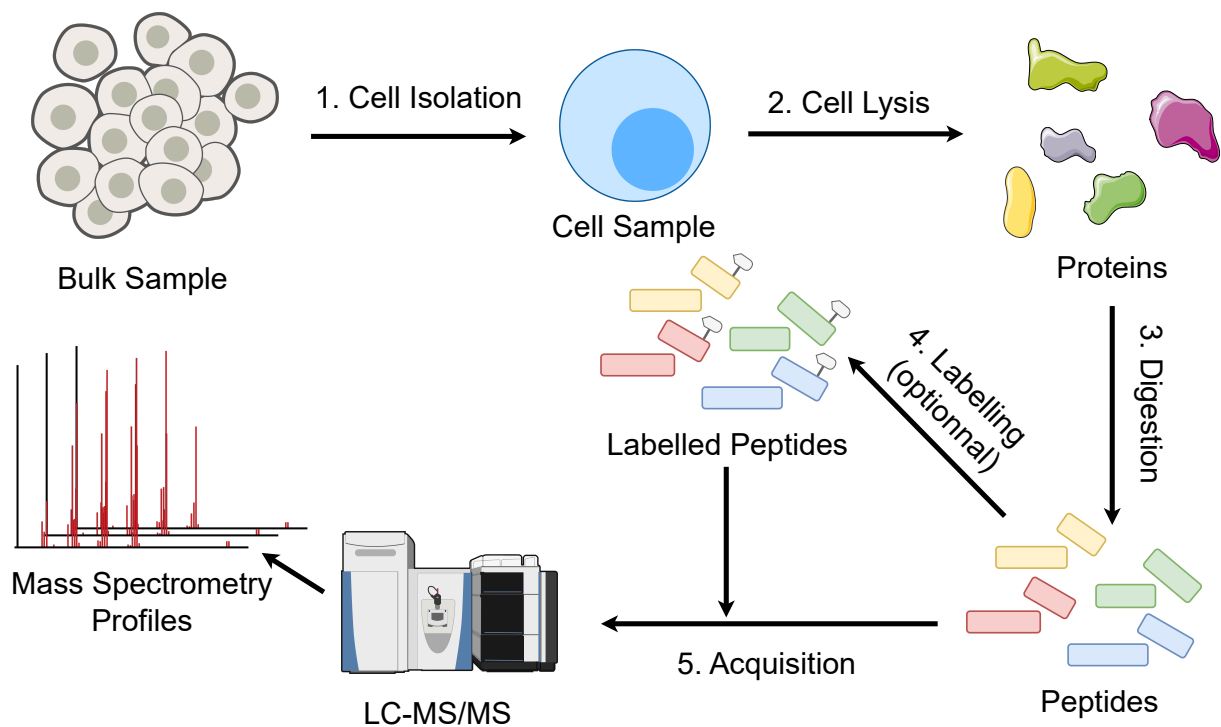


Figure 1. Scheme of a simplified SCP experimental workflow. 1. *Cell Isolation* using capillary aspiration, LCM, FACS or image-based methods. 2. *Cell Lysis* lysis using chemical or mechanical MS-compatible methods. 3. *Proteins digestion* digested using Trypsin and/or Lys-C. 4. *Peptides Labelling* peptides can be labelled using for instance TMT. 5. *Acquisition* processed samples are acquired using a combination of liquid chromatography and mass spectrometry.

damental SCoPE-MS design by optimizing sample preparation, introducing a reference channel, and increasing the multiplexing level to TMT-16. These enhancements enabled quantifying over 3,000 proteins in nearly 1,500 individual cells, greatly improving throughput [17, 31].

At the same time, some labs shifted from DDA to data-independent acquisition (DIA) for increased completeness and data reproducibility. Thielert et al. set up mDIA, a method that combined DIA with multiplexing (TMT-5plex), quantifying as many as 4,000 proteins in 500 single cells [32]. And Derks et al. developed plexDIA a 3 plex method that allowed the quantification of 8,000 proteins [27]. The combination of DIA and multiplexed labelling is a significant milestone in increasing sensitivity and scalability.

In the future, unpublished work by collaborators report a scale up to achieve the acquisition of 50,000 cells, potentially opening the door to a new era of high-throughput single-cell proteomics.

These incremental advances highlight rapid advancement of the field from proof-of-principle to high-throughput, deep-profiling method capable of resolving thousands of proteins in thousands of cells.

MS Method	Labelling	Acquisition Method	Cells	Proteins per cell	Year	Reference
OAD	Label-Free	DDA	100	50	2018	[29]
nanoPOTS (first generation)	Label-Free	DDA	140	330	2018	[25]
SCoPE-MS	Labelled (6)	DDA	190	1500	2018	[3]
nanoPOTS + iBASIL	Labelled (13)	DDA	100	1500	2020	[30]
SCoPE-2	Labelled (16)	DDA	1490	3000	2022	[17, 31]
mDIA	Labelled (3)	DIA	500	4000	2022	[32]
Not yet published	Labelled	DDA	~50 000	~2500	2025-26	/

Table 1. Non exhaustive list of MS methods with their associated labelling strategy (number of cell per run in parenthesis), acquisition method, the number of cells acquired in the original paper, the average number of detected proteins per cell in the original paper, the year of publication and the reference to the original paper.

1.2. Software Infrastructure

The Place of Software in Research

Software plays a central role in modern scientific research. Often, it is developed as part of a specific scientific project to address particular analytical needs. More and more, journals require not only open data but also open and accessible code as a condition for publication, aiming to promote transparency and reproducibility [33]. However, such software is frequently considered a secondary outcome, often poorly documented and commonly abandoned shortly after the associated paper is published [34]. This approach undermines the long-term value of computational tools in advancing science.

From a bioinformatics perspective, it is crucial not to let the race for publishing scientific papers take precedence over the development of robust and sustainable software. Building high-quality software should be seen as a valuable scientific contribution in itself. Such software not only supports rigorous and reproducible research but also enables other scientists to build upon existing work. Moreover, the impact of software tools could be recognised and incorporated into the metrics used to evaluate scientific contributions, encouraging researchers to invest in creating reliable and widely usable softwares [35].

A high-quality scientific software should enhance reproducibility by providing clear, structured workflows that allow users to retrace each step of an analysis. It should be accessible, open source, well documented, and approachable enough for users who may not be expert programmers. Maintenance is also essential: software must be updated to stay compatible with evolving data formats, dependencies, and user needs. Furthermore, performance is a key consideration; software should be computationally efficient enough to handle realistic datasets without becoming a bottleneck. Reliability is equally critical, software should be rigorously tested to minimise bugs or errors that could compromise the validity of results [36, 37]. Finally, good software should be designed with scalability in mind, capable of adapting to increasingly large and complex data as experimental technologies evolve.

Importantly, performance optimisation must be balanced with software quality. Excessive focus on computational speed in early development can increase the code base complexity, potentially hindering maintainability, new feature development, usability, and sometimes reproducibility (ie. non-deterministic algorithm to speed-up computation). Optimisation efforts should therefore be driven by actual user needs [38]. This balance ensures scientific software remains fast, reliable, and adaptable, ultimately supporting sustainable and impactful research.

By adhering to these principles, scientific software becomes more than a tool for one project, it becomes an integral part of a transparent, collaborative, and reproducible research ecosystem.

End-to-end SCP data processing

The processing of data generated from mass spectrometry-based single-cell proteomics (SCP) experiments involves a series of analytical steps, starting from raw instrument acquisition to final statistical analysis. This global pipeline can be divided into several key steps.

It is important to note that most of the computational tools currently used in SCP were originally developed for bulk proteomics data analysis. Despite this, they have been successfully adapted to single-cell workflows, though this often involves additional challenges related to lower input amounts and increased noise.

After the acquisition, the first key step is the identification of precursors, which involves matching MS/MS spectra to peptide sequences using search engines. Popular tools for this step include *MaxQuant* [39], *MSFragger* [40], *Sage* [41], and *DIA-NN* [42], each tailored to specific acquisition strategies (DDA or DIA).

Following identification, quantification estimates the abundance of each precursor in each sample by measuring the intensity of MS signals from precursor or fragment ions. In most workflows, the same software used for identification also performs quantification.

Quantification is closely related to aggregation, the process of summarising measurements from the peptide–spectrum match (PSM) level to peptide-level quantities, and then to protein-level quantities. A critical consideration during aggregation is the handling of shared peptide sequences that map to more than one protein. These can arise from homologous proteins, protein isoforms, or conserved sequence regions. Shared peptides introduce ambiguity, as their measured intensity cannot be unambiguously assigned to a single protein, and naive inclusion can inflate abundance estimates. Common strategies include excluding shared peptides entirely, assigning them proportionally based on unique peptide intensities, or modelling their contribution explicitly in a protein inference framework [43]. In addition to shared peptides, careful treatment of missing values and the choice of summarisation strategy, such as summing, taking the median of feature intensities, or applying more advanced statistical models [44] is required. The choice and order of these aggregation steps, particularly in the context of other preprocessing operations like quality control, can influence final protein quantification and, therefore, downstream biological interpretation.

After quantification, the raw quantitative data undergoes preprocessing, which typically includes log transformation, normalisation (to correct systematic biases across samples/features), batch effect correction (to account for variability introduced by sample preparation or multiplexing batches), and filtering of low-quality features or cells [45]. Note that these operation can be used on the different data levels, before aggregation.

It is worth noting that different labs often implement different workflows for these steps, using varied tools, parameters, and filtering strategies. These differences can lead to substantial variation in the final results, underscoring the importance of moving toward more standardised, transparent and reproducible workflows for SCP data analysis [46].

Finally, the pipeline generally concludes with statistical analysis and visualization, which aim to identify differential protein abundance, detect cellular subpopulations, or infer biological trajectories. Common techniques include clustering, dimensionality reduction (PCA, UMAP, t-SNE), and statistical modelling specific to proteomics data.

Downstream data processing is frequently performed using the R programming language or python, which provide powerful tools for statistical analysis and visualization. Specialized R packages such as *QFeatures* [47] and *scp* [46] offer tailored infrastructures for organizing, manipulating, and analysing single-cell proteomics datasets.

The R/Bioconductor ecosystem for SCP quantitative data analysis

Bioconductor is a widely used and accepted open-source project that host an extensible and integrated ecosystem for the analysis of high-throughput omics data within the R statistical programming language. The philosophy of Bioconductor is centered on *interoperability*, *modularity*, and *reproducibility*. These are achieved through standardised data formats and uniform programming interfaces, facilitating package integration to produce reproducible analysis workflows.

SummarizedExperiment At the center of Bioconductor's design philosophy are data containers that allow the representation of complex experimental data. The *SummarizedExperiment* class (SE) [48] is the most used container. A *SummarizedExperiment* object is divided into 3 main parts: the *assay*, the *rowData* and the *colData* (Figure 2). The *assay* contains the quantitative data in a matrix format. The rows of this matrix represent features (in SCP: PSMs, peptides, proteins) and the columns represent the different samples (in SCP: single-cells), therefore each value represent the intensity of a feature in a single-cell. The second part, the *rowData* contains

the metadata associated with each features. In the SCP context this data can for instance be the identification scores, peptide sequences or other metrics returned during the identification step. Finally, the *colData* part contains the experimental design information, the metadata associated with each sample. This data can for instance be the patient, treatment or cell type associated with the sample.

The *SingleCellExperiment* (SCE) class [49] is an extension of the *SummarizedExperiment* container designed to work with single cell data, it adds single-cell specific infrastructure such as reduced dimensional representations (e.g., PCA, UMAP) specific subcontainer.

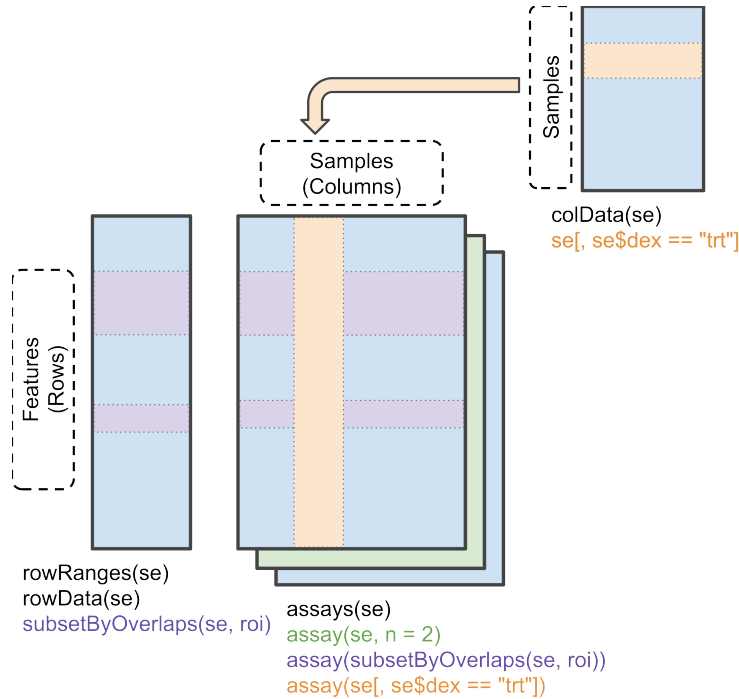


Figure 2. Scheme of the *SummarizedExperiment* class. *assay* is represented at the center, *rowData* on the left and *colData* on the upper right. The associated accessors are also annotated. Figure from the *SummarizedExperiment* official documentation [48].

QFeatures The *QFeatures* R package plays a central role in the analysis of mass spectrometry-based proteomics data. It was originally developed for bulk proteomics but it is now extended for SCP, thanks to the *scp* package (cf. next section). *QFeatures* was designed to manage and analyse quantitative data acquired in MS experiments and keep links between the different data levels (Figure 3). It is built on top of the *MultiAssayExperiment* class, which is designed to contain several SE and add tools to handle them in a easy way. The *QFeatures* class provides an set-oriented framework where each set maps to an unique data acquisition or to a different step in an analysis workflow¹. Each set is represented as a *SummarizedExperiment* object. The main advantage of this infrastructure, is the support of explicit set association trough links between sets. With these links, the users can monitor how features are aggregated from lower to higher levels (e.g., from PSMs through peptides to proteins) This can then be used to make operation on the protein level based on associated PSMs data. This set philosophy also allow the traceability of the different processing steps, enabling reproducibility, indeed each version of the data (coming from a processing step) can be stored in new set that are linked together.

¹Historically, the *QFeatures* package documentation used "assay" to name each *SummarizedExperiment* present in the *QFeatures* object. But to avoid confusion between *QFeatures* assays (individual SE) and *SummarizedExperiment assay* slot (quantitative matrix), this nomenclature was changed to use "set".

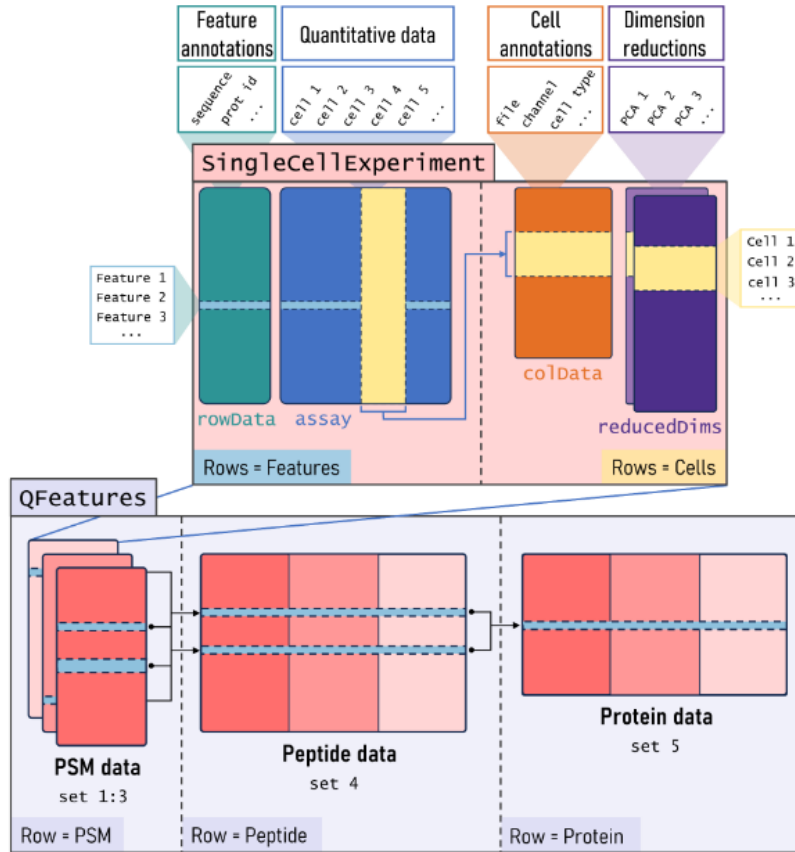


Figure 3. Scheme of the *QFeatures* infrastructure in the context of SCP data. Each assay (red rectangle) is a *SingleCellExperiment* object, the top part of the figure is a representation of the structure of a SCE object. Each assay can be linked to another for instance between the PSM, peptide and protein level. There is a row mapping between linked assays. Figure from Grégoire et al. [45]

scp To further specialise *QFeatures* for SCP data, the *scp* package has been developed. It is built on top of *QFeatures* with tools that are specific for single-cell proteomics pipelines. In particular, *scp* uses the *SingleCellExperiment* class as the internal data representation for a set, thus being compatible with many downstream packages from the single-cell analysis Bioconductor ecosystem (Figure 3). Also, *scp* incorporates built-in operations to process the data such as quality control, filtering, normalization, batch correction, and visualization, all of which have been optimized for SCP data.

Together, *QFeatures* and *scp* promote workflow reproducibility in a field where practices are still highly variable and fractured between labs [46]. By applying structured, modular tools within the Bioconductor paradigm, researchers achieve more *reproducibility*, *data traceability*, and *ease of collaboration*.

These packages do not exist in isolation, they integrate perfectly well within the Bioconductor ecosystem. For instance, the *msqrob2* package [50] can be paired with *QFeatures* and *scp* to enable effective statistical modelling and differential expression. Similarly, packages of visualization, dimensionality reduction, and annotation can be used into the analysis pipeline with ease.

Operations on *QFeatures* objects We present here the principal types of operations that can be performed on a *QFeatures* object. These operations can be grouped into two main classes: *per set operations*, that are applied independently to each set (i.e., each *SummarizedExperiment* object), and *global operations*, that work across multiple sets at once (Figure 4).

Per set operations These operations are applied individually on each set stored in a *QFeatures* object, and leverage the structure of the *SummarizedExperiment*, including the *assay*, *rowData*, and *colData* slots:

- **Computational operations:** These involve mathematical or statistical transformations applied directly to the quantitative data matrix (the *assay*). They do not conceptually alter or use the *rowData* or *colData*. Examples include log-transformation, normalization, or imputation.
- **Row-wise filtering:** This type of operation filters features (rows of the quantitative matrix) based on conditions defined in the *rowData*. Typically used for quality control, for instance to filter out low-confidence PSMs, peptides, or proteins. After filtering, both the number of rows in the assay and the corresponding entries in *rowData* are reduced.
- **Column-wise filtering:** Similar to row-wise filtering, but applied to the columns of the assay (e.g., single cells), based on metadata found in the *colData*. This is often used to remove low-quality cells. Both the number of columns in the assay and the corresponding entries in the *colData* are reduced.
- **Feature aggregation:** This operation merges multiple features into a single one across all samples, using a grouping variable from *rowData* (e.g., to aggregate PSMs into peptides, or peptides into proteins). Aggregation is performed using a user-specified summarization function (e.g., mean, median).

For each metadata variable in the *rowData*, the aggregation process retains only those that are invariant, that is, variables that have the same value across all features within a group. Variables that differ within a group are discarded, as they cannot be unambiguously assigned to the aggregated feature.

Note that a more complex alternative aggregation method is the *robustSummary*, which fits a robust linear model to the feature intensities within each group using the `r1m` function from the *MASS* package. This approach is less sensitive to outliers and noise, providing a more reliable summary statistic by down-weighting extreme values during the estimation of the aggregated feature intensity.

Global operation Global operations act across multiple sets within a *QFeatures* object. A primary example is:

- **Set joining (`joinAssays`):** This operation merges several sets into a single new set. It simultaneously uses the *assay*, *rowData*, and *colData* components of each input set. The resulting set includes all samples (columns) and features (rows) coming in the original sets. Features present in multiple assays are aligned and merged, while features unique to a single assay are retained with missing values filled where appropriate. This operation enables easy downstream analysis on a specific features level.

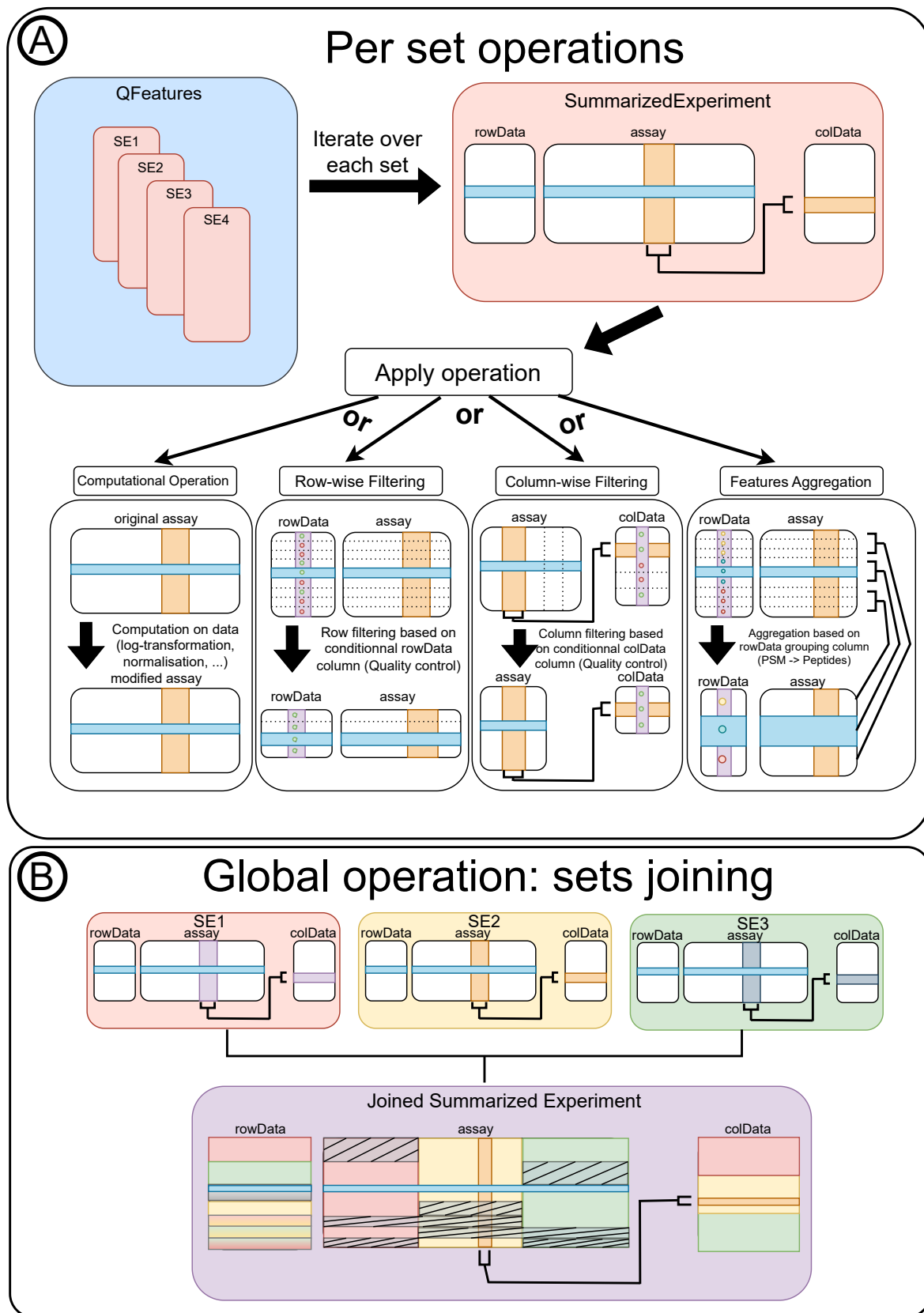


Figure 4. Overview of the main operation types performed on a *QFeatures* object. **A:** Per set operations are applied individually to each set and include computational transformation, row/column filtering, and feature aggregation. **B:** The global operation shows the joining of different sets into a unified *SummarizedExperiment*. Note that regions in crossed lines are regions with missing values, due to the presence of specific features in only a portion of sets. This aspect is also reflected in the *rowData* colouring, indeed grey colour indicate that the feature is present in all sets, whereas coloured regions indicate from which sets the features comes from (multicolor regions indicate features coming from several sets).

Modelling SCP data

Mass spectrometry-based single-cell proteomics produces highly sparse and noisy data due to the extremely low quantities of starting material and the technical limitations of current instrumentation. Unlike single-cell transcriptomics, proteins cannot be amplified, resulting in frequent missing values and increased measurement variability. Effective computational analysis is therefore essential to extract meaningful biological insights from SCP experiments.

Several dedicated statistical and computational frameworks have been developed to address these challenges. Below we describe two widely used tools: *scplainer*, that was developed specifically for SCP data, and *msqrob2*, that was originally designed for bulk proteomics but that could also be applied to single-cell proteomics. We also briefly highlight other available approaches.

scplainer *scplainer* is a dedicated statistical framework for modelling SCP data [51]. SCP data are highly sparse, noisy, and affected by batch effects, making analysis particularly challenging. *scplainer* addresses these issues by combining minimal, well-justified preprocessing with a simple and flexible linear modelling to disentangle technical from biological variability.

Unlike pipelines that sequentially apply imputation, batch correction, and normalisation—which can introduce biases and obscure uncertainty, *scplainer* integrates these steps into a single modelling process. The method takes peptide-level intensity data and user-provided experimental descriptors (e.g., cell type, treatment, acquisition batch), then fits peptide-wise linear regression models. This approach allows explicit modelling of known technical and biological factors, enabling effective batch correction while preserving biological signals.

The estimated model coefficients are highly interpretable: categorical descriptors yield fold-change estimates between groups. *scplainer* also provides tools for analysis of variance (to quantify the contribution of each factor), differential abundance testing, and component analysis to explore latent biological structure in the data.

A key advantage is that *scplainer* avoids imputation by handling missing values directly in the modelling. It implements an objective filtering criterion based on the ratio of observations to model parameters, ensuring peptides with insufficient data are excluded without arbitrary missing-value thresholds. By doing so, it reduces bias and preserves the inherent uncertainty of SCP data.

scplainer is implemented within the *scp* package.

msqrob2 *msqrob2* is a statistical framework that was designed to improve differential abundance estimation in label-free shotgun proteomics by modeling data at the peptide level [52]. Classical analysis pipelines typically summarize peptide intensities to obtain protein-level values before testing for differential abundance. While simple, this approach can discard valuable information and yield biased fold-change estimates, particularly in sparse datasets with many missing peptide intensities.

msqrob2 instead fits mixed linear regression models at the peptide-level intensities, estimating treatment effects while accounting for peptide-specific and technical variation. However, naive peptide-based models can suffer from overfitting, unstable variance estimates, and sensitivity to outlying peptide measurements. To address these challenges, *msqrob2* introduces three key improvements:

- **Ridge regression:** By applying a penalty to regression coefficients, ridge regression shrinks parameter estimates, reducing variance and protecting against overfitting—especially for proteins identified by only a few peptides.
- **Empirical Bayes variance estimation:** Variance estimates are stabilized by borrowing strength across proteins. Protein-specific residual variances are shrunk toward a global prior,

reducing false positives caused by unrealistically small variance estimates in sparse data.

- **M-estimation with Huber weights:** Outlier peptide intensities are downweighted using robust regression techniques, mitigating their impact on fold-change estimates.

Although originally developed for bulk label-free proteomics, *msqrob2* could be applied to MS-based single-cell proteomics data, where sparsity and variability are even more pronounced. Its robust statistical modelling makes it a potential valuable tool for extracting reliable quantitative insights from challenging SCP datasets. Note that *msqrob2* can also be used to directly model protein intensities.

Other approaches Beyond dedicated single-cell proteomics tools, several other methods could be used in the context of SCP, we here describe a few of them. *Triqler* [53] is a Bayesian graphical model that integrates identification and quantification uncertainties, propagates error throughout the analysis, and provides posterior probabilities for differential expression while controlling the false discovery rate. Another alternative is the new method *limpa* that is based on the popular *limma* method used in RNAseq differential analysis [54]. This new method is designed to improve quantification and differential expression analysis in MS proteomics data. It addresses the problem of missing values by implementing probabilistic information recovery: peptide-level measurements with more missingness are treated as less precise and are downweighted accordingly. This approach preserves statistical power while reducing false discoveries.

An unexplored alternative: pseudobulking

Pseudobulking is a method that is used in scRNAseq. It consists of aggregating quantitative features values belonging to cells from a same patient and cell subpopulation. This approach allows to reduce the data complexity and sparseness while keeping biological meaning information (i.e. variation between patient and cell subpopulation). Then bulk modelling methods can be applied to this (*pseudobulk*-)data. This method has been explored in scRNAseq and showed great performance [23, 55, 56, 57].

Currently, multi-patients SCP experiments are not very common. This is due to the fact that SCP is still in its development phase, and the number of acquired cells per experiments is still limited. However, as the field continues to evolve, we expect to have more and more experiment with a multi-patients experimental design. For instance Furtwängler et al. [58], explored the human hematopoietic stem and progenitor cell hierarchy by sampling cells from six donors. Unpublished work from collaborators, measured cells from five different patients at different time points. Using a multi-patients setup is crucial to be able to answer with confidence to biological questions. Indeed, using different biological replicates we are able to remove the inter-patients effect from a condition of interest. In this context, pseudobulking could be used to model future SCP data, which would involve more biological replicates.

1.3. Current challenges in SCP

Mass spectrometry-based single-cell proteomics is a rapidly advancing field, but it faces several important experimental and analytical challenges that must be addressed to ensure robust and biologically meaningful results.

Experimental challenges From an experimental point of view, one of the major challenges is the number of cells analysed. To obtain sufficient statistical power and ensure robust representation across diverse cell types, including rare populations, a large number of cells must be processed. This exact number will depend on the expected frequency of the cell types of interest, which means that studies aiming to profile rare cell populations must scale up substantially [59]. Another essential factor is the proteomic depth, adequate depth is required to detect enough proteins per cell to allow accurate annotation and biological interpretation. Shallow proteomic coverage

can lead to the missing of key protein markers, leading to incomplete or misleading biological conclusions. A third challenge is the presence of batch effects, a form of technical noise that introduces unwanted variability across samples. Batch effects can arise from differences in sample preparation protocols or variations in instrument performance across runs. These effects can distort the true biological signals unless properly accounted for during the data analysis. Although spike-in standards are common in the clinical proteomics field [60], they are not yet applied to SCP and will maybe not ever be applicable due to the low volumes used in SCP. Also, capturing biological variability requires analysing samples from different patients, which adds another layer of complexity at the experimental and analysis steps. But this is necessary for translating SCP findings to meaningful clinical or biological insights.

Computational challenges From a computational perspective, current quantitative analysis software also faces significant limitations. First, SCP data is characterised by high levels of missingness due to the low protein abundance and stochastic sampling at the single-cell level. This requires tools specifically designed to handle missing data appropriately, avoiding biases (imputation) and loss of information [61]. Additionally, as both the number of cells and the proteomic depth increase, the resulting datasets grow rapidly in size, and could exceed the capacity of standard computing environments. While such large datasets are not yet the norm but rather the exception, we can anticipate that increasingly large datasets will become more common in the near future. These big datasets can be challenging to load into memory and require long processing times, creating significant bottlenecks in analysis. This problem is expected to become even more pronounced as SCP technology evolves and dataset sizes continue to grow (as underlined in the *History and technical evolution of SCP* section). Furthermore, existing modelling tools must be adapted, or new ones developed, to properly accommodate the unique characteristics of SCP data. These include handling missing values, complex experimental designs involving multiple factors (e.g., cell type, patient, treatment), and technical confounders (batch effects).

In addition, there is currently substantial variability in data analysis workflows across different laboratories. This variability stems not only from differences in experimental protocols but also from the absence of consensus on key processing steps which are often implemented in different orders and with different methods (quality control, normalisation, batch correction, ...). This often leads to highly divergent results even from similar datasets [46]. This lack of standardisation harms reproducibility and comparability across studies. Therefore, promoting reproducibility and transparency in workflow development is essential to build trust in findings and to enable meaningful biological interpretation. Software infrastructure like *QFeatures* and *scp* can help to make the quantitative data processing more reproducible by tracking each step of the analysis.

Addressing these challenges at both the analysis and experiment levels is crucial for fully realising the potential of SCP in biological research.

2. Research Questions

This thesis aims to address two key challenges in the analysis of mass spectrometry-based single-cell proteomics data: computational scalability and data modelling. To this end, the work is divided into two parts, each guided by a central research question.

- **Part I – Computational Framework Optimization:**

How can the current R/Bioconductor SCP framework be optimized to handle larger datasets and perform analyses more efficiently?

This part investigates the performance limitations of existing R-based tools for SCP (i.e. *QFeatures* and *scp*) and explores potential strategies to improve their scalability, memory usage, and computation speed, with the goal of enabling faster and more efficient analysis as dataset sizes continue to grow.

- **Part II – Modeling Approaches:**

Can pseudobulking, a method commonly used in single-cell transcriptomics, be effectively applied to SCP data, and how does its performance compare to existing modeling approaches in SCP?

This part evaluates whether pseudobulking, a strategy that aggregates cell-level data to reduce noise and improve statistical power, can be used on SCP data. It also benchmarks its performance against established modelling tools (scplainer [51], msqrob2 [52]).

3. Methodology and Thesis Outlines

This thesis combines software optimisation, simulation, and benchmarking (computational ² and technical³) to address the two research questions outlined previously. The methodology is divided into two main components:

- **Technical benchmarking and performance optimization:** To optimize the current R/Bioconductor SCP framework, we conduct reproducible performance benchmarks using both real and simulated datasets. Profiling tools are employed to systematically identify computational bottlenecks across different stages of the analysis pipeline. Once bottlenecks are identified, targeted optimisations are implemented. The effectiveness of each optimisation is evaluated through controlled benchmarking procedures.
- **Computational benchmarking of the pseudobulking method through simulation:** To evaluate the suitability of pseudobulking for SCP, we simulate multi-patient, multi-condition datasets. Simulated datasets include introduced known condition-specific changes to establish a ground truth, allowing for objective performance assessment. Benchmarking compares the pseudobulking strategy to standard SCP modeling approaches under various conditions (i.e. different number of patients/cells and different effect strength).

²Computational benchmarking involves assessing the performance of different data processing methods to determine which one is the most suited for a specific application.

³Technical benchmarking refers to evaluating the performance of software algorithms or workflows, such as execution time, memory usage, and scalability, focusing on code efficiency.

Materials and Methods

1. Datasets

1.1. Leduc et al. 2022

Leduc et al. [16] present an experimental workflow based on the SCoPE2 protocol [17, 31]. Using a TMT-18 labelling procedure, they acquired 1543 cells across 134 different runs. The dataset achieves great proteomic depth, with a total of 2800 proteins identified. In each run, monocytes and melanoma cells were analysed. A carrier sample was also used to boost peptide detection. Raw data was processed using the MaxQuant software [39] to obtain PSM-level data. The PSM-level data was then loaded into R as a *QFeatures* object; as the dataset uses multiplexing, each set (134 in total) contains 18 cells.

We choose this dataset because it is a well-known and thoroughly documented reference in the single-cell proteomics field. It is used as our reference multiplexed dataset in our analyses.

1.2. Brunner et al. 2022

Brunner et al. [15] used label-free SCP to measure protein abundance in 430 HeLa cells. The dataset contains cells arrested at different cell cycle stages using drugs. Thousands of proteins were identified per cell. MaxQuant was used to process the raw data. The PSM-level quantitative data was loaded into R as a *QFeatures* object. Since this dataset was acquired using a label-free technique, each set contains only one cell (one column per set).

We choose this dataset because it is a well-known dataset and it is easily accessible through the *scpdata* R package. It is used as our reference label-free dataset in our analyses.

1.3. Bulk HeLa control samples (Webel et al. 2024)

Webel et al. [62] collected over 7444 bulk proteomics MS acquisitions. These acquisitions come from quality control and maintenance test, in which HeLa samples were used. These come from different laboratories, and instruments and span during several years. MaxQuant was used on each run to identify and quantify the PSMs. Since these are bulk samples, the proteome depth is greater than for single cell sample, only runs with at least 15 000 identified peptides were kept (7444). The 7444 files that contain the MaxQuant PSM output are available publicly. This dataset, is therefore quite large with thousands of samples and over 15 000 peptides identified per sample. Since this dataset acquired the samples using independent runs, each set contains one sample (one column per set).

We choose this dataset because it represents a challenging, large-scale dataset with far more data than typical standard datasets, which can usually be loaded into memory on a standard laptop (<16GB RAM). In contrast, this dataset's size exceeds typical memory constraints, providing a valuable benchmark for computational scalability.

2. Benchmarks

In this section, we present a series of benchmarks designed to evaluate the computational performance of the *QFeatures* and *scp* packages under different scenarios relevant to single-cell proteomics analysis. These benchmarks aim to assess scalability, memory consumption, and runtime efficiency across different data sizes, processing steps, and data structures.

We start with a **real pipeline benchmark** (Section 2.1), in which we reproduce the complete SCP analysis workflow proposed by Leduc et al. [16] on datasets of increasing size. This benchmark provides an end-to-end view of how the infrastructure performs under realistic conditions.

To better isolate bottlenecks, we then perform a **component-level benchmark** (Section 2.2),

where key functions from the *QFeatures* and *scp* packages, used throughout typical analysis workflows—are tested individually. This allows us to identify the most resource-intensive operations independently of a full pipeline context.

We also conduct a **QFeatures memory consumption benchmark** (Section 2.3) to understand how the memory footprint of a *QFeatures* object grows when varying key parameters such as the number of sets, cells per set, features per cell, and number of metadata variables.

Finally, we include a **challenging dataset benchmark** (Section 2.4) using the Weibel et al. HeLa bulk dataset [62], to explore performance on a particularly challenging use case. This benchmark investigates how metadata size, specifically the size of the *rowData*, affects processing time and memory usage in large-scale data loading and manipulation.

All benchmarks were run on a Linux machine with the following specifications:

Machine: Linux (6.8.0-62-generic)

RAM: 134.8 GB

CPU: 36 core(s) - Intel(R) Core(TM) i9-10980XE CPU @ 3.00GHz

2.1. Real Pipeline Benchmark

A first approach to test the performances of *QFeatures* and *scp* was to benchmark these packages in a real case setup. To do so, we used the analysis pipeline presented by Leduc et al. on their dataset [16]. This pipeline has been reproduced in R using the Bioconductor SCP framework by Vanderaa et al. [63]. We will here reuse this pipeline on different dataset size, by generating more cells from the original data.

Analysis pipeline The analysis pipeline reproduced from Leduc et al. is structured to reflect the layered nature of mass spectrometry-based single-cell proteomics data. It progresses sequentially from the raw peptide-spectrum match (PSM) level through peptide-level processing and finally to protein-level refinement. Each stage includes specific operations that address data quality, normalization, aggregation, and statistical correction. This modular structure not only mirrors common practice in SCP data analysis, but also allows benchmarking of each individual step within the *QFeatures* and *scp* frameworks. Below, we outline the principal processing steps involved at each data level, along with their operation category (as defined in the introduction 1.2.):

- **PSM-level processing:**

- **filterPSM** (row-wise filtering): Filter PSMs based on different metrics (e.g., contaminants, low-confidence identifications).
- **filterSCR** (row-wise filtering): Compute the sample-to-carrier ratio for each PSM and filter based on this metric.
- **filterSumSignal** (row-wise filtering): Filter PSMs with no signal in single-cell samples.
- **divideByReference** (computational operation): Normalize by dividing by the reference sample to correct for run-to-run variation.
- **aggregatePSM** (feature aggregation): Aggregate PSMs into peptides.

- **Peptide-level processing:**

- **consensusMap** (row-wise operation): Perform consensus mapping of peptides to their corresponding proteins.
- **zeroToNA** (computational operation): Convert zero values to missing (*NA*) values.

- **joinPeptide** (global operation): Join peptide-level sets into one unified set.
- **filterCV** (column-wise filtering): Filter single cells based on their median coefficient of variation.
- **normPeptide** (computational operation): Normalize peptide-level intensities.
- **filterNA** (column-wise filtering): Filter single cells with excessive missingness ($> 99\%$).
- **logTransformation** (computational operation): Apply log-transformation to the data.
- **aggregatePeptide** (feature aggregation): Aggregate peptides into proteins.
- **Protein-level processing:**
 - **normProt1** (computational operation): Normalize at the protein level.
 - **impute** (computational operation): Impute missing values.
 - **removeBatch** (computational operation): Correct for batch (run) effects.
 - **normProt2** (computational operation): Re-normalize after batch correction.

We report in Figure 5 the resulting QFeatures structure.

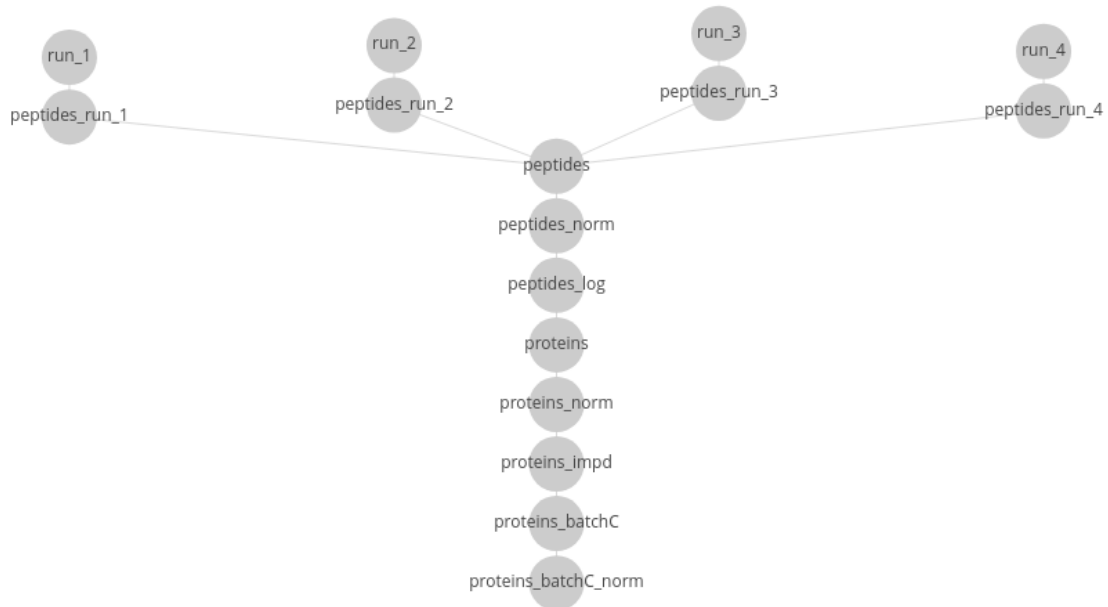


Figure 5. Scheme of the structure of the resulting QFeatures object obtained at the end of the Leduc et al. pipeline. Each circle represent one set. For illustration purpose, we only show here a QFeatures object containing 72 cells. The number of starting sets can be obtained by dividing the number of cells per 18.

Cell generation In order to see how the current infrastructure scale as we increase the number of cells analysed, we generated synthetic cells from the original data. This generation was made using a resampling the existing sets to create new sets. We applied this method to generate new datasets containing: 1000, 2000, 4000, 8000 and 16000 cells.

Benchmark For each data input size, we benchmarked each individual steps of the pipeline (as describe above). We measured 3 metrics per step: the runtime, the random access memory

(RAM) consumption and the maximum of RAM used during the execution (stochastic measure). To account for technical variability, we replicated each measurement 3 times. Finally, we also report the final in memory *QFeatures* object size.

2.2. Individual Components Benchmark

While the previous benchmark (Section 2.1) assessed the performance of a full analysis pipeline, the operations involved in each step rely on a defined set of core functions from the *QFeatures* and *scp* packages. In this section, we isolate and benchmark these individual functions to better understand their computational cost and memory usage independently of the pipeline context.

To find which functions from *QFeatures* and *scp* are the most time- or memory-consuming, we ran a benchmark focusing on a set of individual functions. Each function was benchmarked independently using datasets of increasing size.

The functions selected were based on the minimal SCP workflow proposed by Grégoire et al. [45]. Functions involved in this workflow were identified and benchmarked. Below is the list of benchmarked functions, along with their associated operation category:

- `filterFeatures` (row-wise filtering) – Filter the *QFeatures* object based on variables in the `rowData` slot.
- `subsetByColData` (column-wise filtering) – Filter the *QFeatures* object based on variables in the `colData` slot.
- `zeroIsNA` (computational operation) – Replace zero values in the quantitative data with NA to indicate missing values.
- `aggregateFeaturesOverAssays` (feature aggregation) – Aggregate features across multiple sets within a *QFeatures* object, using a grouping variable.
- `joinAssays` (global operation) – Join several sets into a single set, combining their features and samples.
- `sweep` (computational operation) – Normalise the quantitative data using per-cell or per-feature statistics.
- `logTransform` (computational operation) – Apply $\log_2$ transformation to the quantitative data.
- `aggregateFeatures` (feature aggregation) – Aggregate features within a single set using a grouping column from the `rowData`.
- `impute` (computational operation) – Perform imputation of missing values in the quantitative data.

To be able to observe the impact of the input size, we generate different new objects based on the Leduc et al. 2022 dataset [16]. The procedure used to generate this data is similar to the one used in the Leduc et al. 2022 benchmark. For this benchmark, different dataset for the different level (PSMs, peptides, proteins) are needed, the peptide and protein dataset are constructed by the aggregation of the generated PSM data. We generate for each data level, 4 datasets with the following numbers of cells: 500, 1000, 2000, 4000.

For each function and each dataset size, we report the following metrics: runtime, memory consumption, maximum memory used during execution, *QFeatures* component (`rowData`, `colData`, `assay`) sizes before and after operation (computed by summing the sizes for each *SingleCellExperiment*). Each measurement was repeated 3 times to account for technical variation.

2.3. QFeatures Object Memory Usage Benchmark

To understand how the size of a *QFeatures* object varies when key data parameters are modified, we designed a benchmark focused on four variables known to influence dataset size: the number of sets (runs), the number of cells per set, the number of features acquired per cell, and the number of variables in the `rowData`. Table 2 summarizes the values used for each parameter. In total, 162 unique *QFeatures* objects were generated by combining these settings.

Parameter	Values
Cell per set	[1, 16, 32]
Number of sets	[8, 16, 32, 64, 128, 256]
Features per Cell	[1000, 2000, 4000]
Number of rowData Variables	[10, 50, 100]

Table 2. Parameter values used to generate the *QFeatures* objects. In total, 162 objects were generated.

To construct these objects, we used the Brunner et al. 2022 dataset [15]. Since the dataset is label-free, we employed a simple random sampling strategy to select cells, features, and `rowData` columns according to the chosen parameters. This approach is appropriate for this benchmark since the focus is solely on evaluating object size, and no downstream data processing is performed.

For each of the 162 parameter combinations, we recorded the total size of the resulting *QFeatures* object in memory (`sizeTotal`), as well as the size contributions from the assay data (quantitative matrix; `sizeAssay`), feature metadata (`sizeRowData`), and sample metadata (`sizeColData`).

2.4. Challenging Dataset Benchmark

To test the current *QFeatures* infrastructure on a challenging dataset, we used the bulk HeLa control samples dataset from Webel et al. [62]. Due to the size of the dataset, we couldn't use all the available samples (7444). Indeed, directly loading these 7444 sets into a *QFeatures* object would lead to an memory consumption of around 250 gb of RAM used. Therefore, we decided to subset the dataset and to use only the first 500 acquisitions. To be able to observe the impact of the `rowData` size on the operations carried on the *QFeatures* object, we applied a filtering on the variables of the `rowData`, filtering out the useless columns in the context of our pipeline (going from 61 to 6 columns). We then loaded two version of the dataset as a *QFeatures*, one with and one without the `rowData` filtering. We then applied a minimal workflow to test the impact of the `rowData` subsetting on the runtime of the different steps. These steps included filtering on sample and features, aggregation and joining.

2.5. VerR

In this work, several optimisation were implemented. To test the impact of the optimisations on key steps, we needed to compare the performances of the different package versions. To be able to compare the package versions in a reproducible way, we developed the *VerR* package. This package, which use the capabilities of the *renv* package and automate them, allows to build isolated environments separated from the global R library in which the a list of user defined packages with specific versions will be installed. The specific package version can be retrieved using *renv* package version repository (CRAN / Bioconductor) or GitHub by specifying commit id. Once environments are created, *VerR* provides function to run scripts or chunks of code in the desired environments, returning the results for easy comparison. The package also provide a wrapper to directly benchmark the runtime of a chunk of code in the different environments. This workflow enabled us to measure the effect of our optimisations in a reproducible manner, isolating

changes to specific package versions. *VerR* is open source, well documented and available on GitHub (<https://github.com/leopoldguyot/VerR>).

3. PseudoBulking

Currently, no available SCP dataset exists that is suited for analysis with pseudobulk methods. To be compatible with pseudobulk strategies, a dataset must contain multiple design layers: distinct cell types (i.e., subpopulations), multiple patients, and a treatment condition of interest.

As such, to evaluate the efficacy of pseudobulking approaches on SCP data, we simulate a suitable dataset. Simulation is necessary for two key reasons: (i) to generate an experimental design where pseudobulk methods are applicable, and (ii) to know the ground truth, that is, which proteins are differentially abundant, enabling performance metrics to be computed.

3.1. Simulated Data Generation

We base our simulation on the Brunner et al. (2022) dataset, chosen for its label-free nature, which simplifies simulation by avoiding the need to explicitly model multi-cells acquisition effects present in multiplexed datasets. In a label-free setting, technical variation is directly present at the cell level (with each cell acquired separately), thus this effect will be implicitly captured in the simulated base protein intensities.

We choose to work at the protein level rather than the peptide level. Although modelling at the peptide level has been shown to improve performance [52], protein-level analysis remains common in current workflows. More importantly, working at the protein level reduces computational complexity, since there are fewer proteins than peptides to model individually. This aspect is important due to the number of datasets that will be simulated and modelled. Moreover, as our goal is to compare pseudobulk methods against alternative modelling strategies, using peptides would not affect this comparative assessment.

The first step of the simulation involves parameter extraction from experimental data. To isolate intrinsic variability, we retain only control cells, resulting in a reduction from 434 to 203 cells. This allows us to capture cell-to-cell variation (biological and technical variation) in the absence of any treatment effect.

We apply a log transformation to the protein intensities to approximate normality [64, 65], then compute for each protein:

- Mean log-intensity μ_i
- Standard deviation σ_i
- Missingness rate π_i

Using these parameters, synthetic log-intensities are generated by sampling from a normal distribution:

$$X_{ij} \sim \mathcal{N}(\mu_i, \sigma_i^2)$$

Missing values are introduced per protein using a Bernoulli sampling process:

$$M_{ij} \sim \text{Bernoulli}(\pi_i) \quad \text{and} \quad X_{ij} = \text{NA} \text{ if } M_{ij} = 1$$

We then introduce three layers of variability by adding effects modelled as normal distributions whose standard deviation is scaled by the protein mean intensity μ_i :

- **Patient effect:** For each of the 16 patients, we apply a protein-wise shift drawn from:

$$\delta_{i,p}^{\text{patient}} \sim \mathcal{N}(0, (0.1 \cdot \mu_i)^2)$$

- **Cell type effect:** Five cell types are simulated. For each, a subset (1/3) of the proteins receive a shift obtained from:

$$\delta_{i,c}^{\text{cell type}} \sim \mathcal{N}(0, (0.2 \cdot \mu_i)^2)$$

- **Treatment effect:** Half of the patients receive a treatment affecting one-third of the proteins. Treated cells are given a fixed shift proportional to the protein's mean intensity.

$$\delta_i^{\text{treatment}} = \theta \cdot \mu_i \quad \text{with } \theta \in \{0.1, 0.15, 0.2, 0.25\}$$

We choose these different treatment shift values to investigate how the strength of this effect impacts model performance. These specific values are chosen because they are on the same order of magnitude as the cell type and patient effects shifts.

The full intensity model becomes:

$$X_{ij} = \mu_i + \epsilon_{ij} + A_i^{\text{patient}} \cdot \delta_{i,p_j}^{\text{patient}} + A_i^{\text{cell type}} \cdot \delta_{i,c_j}^{\text{cell type}} + A_i^{\text{treatment}} \cdot \delta_i^{\text{treatment}} \cdot \mathbb{I}_{t_j=1}$$

Where:

- μ_i : baseline log-intensity of protein i (mean)
- $\epsilon_{ij} \sim \mathcal{N}(0, \sigma_i^2)$: noise term
- $A_i^{\text{patient}} = 1$: all proteins affected by patient effect
- $A_i^{\text{cell type}} \in \{0, 1\}$: one-third of proteins affected by cell type effect
- $A_i^{\text{treatment}} \in \{0, 1\}$: one-third of proteins affected by treatment
- p_j, c_j, t_j : patient, cell type, and treatment assignment of cell j

This simulation results in a dataset with three hierarchical effects (patient, cell type, and treatment). We simulate datasets with varying numbers of cells per (patient, cell type) combination: 10, 25, 50, and 100, resulting in total dataset sizes of 800, 2000, 4000, and 8000 cells, respectively. The simulation process is illustrated in Figure 6.

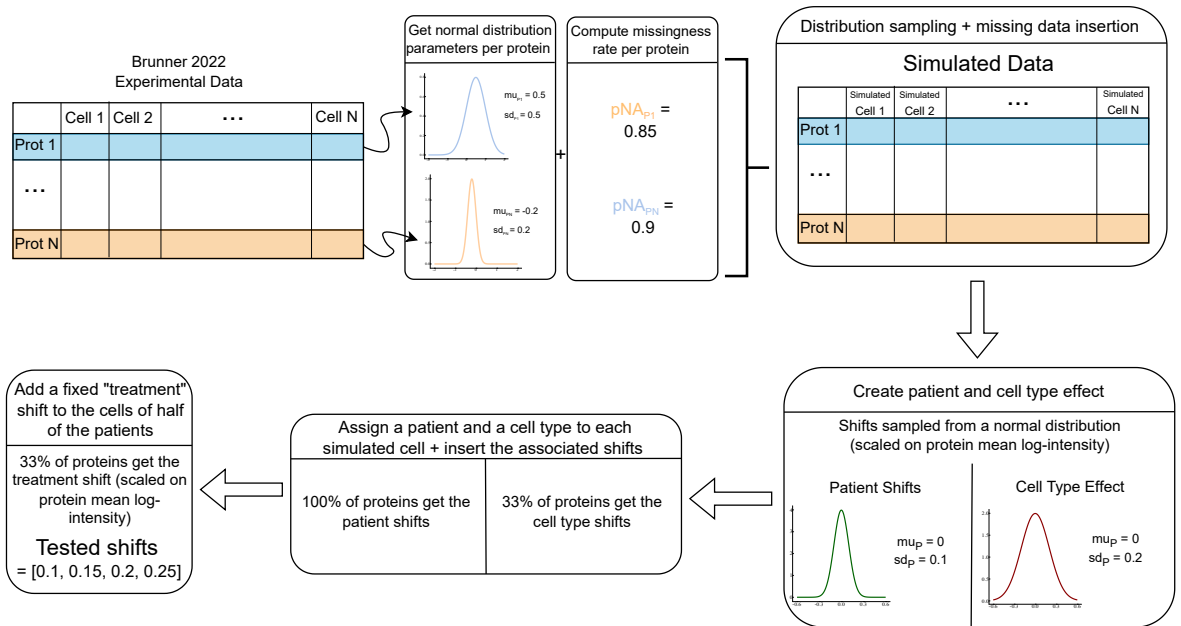


Figure 6. Scheme of the simulation framework.

3.2. Data Aggregation and Modelling

To evaluate the performance of different modelling strategies, we tested six distinct methods. Performance was assessed using the following metrics:

- **True Positive Rate (TPR):**

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

- **False Discovery Rate (FDR):**

$$\text{FDR} = \frac{\text{FP}}{\text{TP} + \text{FP}}$$

These were computed at adjusted p-value thresholds of 0.01, 0.05, and 0.1.

Modeling Methods Tested:

1. **msqrob2:** Fixed effects for cell type and treatment, random effect for patient.
2. **scplainer:** The patient and treatment variables are collinear because knowing the patient automatically determines the treatment. In a fixed-effects framework, this perfect correlation leads to confounding, making it impossible to estimate separate effects for both variables in the same model. Therefore, only cell type and treatment were included as fixed effects, and patient was omitted.
3. **Aggregation + msqrob2:** Data aggregated by (cell type, patient) before fitting msqrob2 using the same model formula as before. Four aggregation methods were evaluated:
 - **Mean:** Arithmetic mean of log-intensities.
 - **Median:** Median of log-intensities.
 - **Sum:** Sum of log-intensities.
 - **robustSummary:** Robust summarisation originally used for PSM/peptide aggregation, adapted here for aggregating across cells. This method helps to mitigate outlier influence [44].

For each model method we report the execution time for the different dataset sizes (parallel parameter is 4 cores for msqrob2). And we repeat the whole process with 3 different seeds for the simulation to ensure that our results are robust.

Results

1. Benchmarks

1.1. Real Pipeline Benchmark

Global trends We first analysed the *peakRAM* [66], which measures the maximum RAM used by a step on top of the memory already occupied by the *QFeatures* object itself. This metric captures transient but critical memory demand. At 16000 cells, the *peakRAM* exceeds 13 GB (Figure 7 A), which is considerable and suggests potential scalability limitations on standard hardware. The relationship between the number of cells and *peakRAM* is not strictly linear, with the curve indicating a slightly quadratic trend. This implies that memory overhead grows faster than the number of cells, likely due to internal data structures or operations that do not scale linearly.

We then examined the total runtime of the pipeline (Figure 7 B). For the largest dataset of 16000 cells, the complete workflow took nearly 100 minutes to run. The increase in runtime is clearly non-linear, approximately quadratic, which indicates that the pipeline contains steps with complexity greater than $O(n)$ with respect to the number of cells.

Regarding the final object size in memory (Figure 7 C), the resulting *QFeatures* object exceeds 10 GB for 16000 cells. Here, the relationship between the number of cells and the object size is linear.

Importantly, when processing 16000 cells, the maximum RAM used at a given time combines the object size (over 10 GB) with the *peakRAM* overhead (over 13 GB), leading to a total of approximately 24 GB of RAM required. Such requirements are demanding for typical laptop, indicating that large scale single-cell proteomics analyses may require workstation or server grade resources.

Finally, it is noteworthy that for both *peakRAM* and runtime, the technical variation across replicates was small (error bars in the figures are negligible), suggesting that the benchmarking results are stable and can be trusted.

Step-wise runtime analysis We also analysed the mean runtime per pipeline step (Figure 8). The most time consuming operations across all dataset sizes were *divideByReference*, *aggregatePSM*, *filterCV*, *impute*, *zeroToNA*, and *joinPeptide*. Among these, *divideByReference*, *impute*, and *filterCV* exhibited a clear non-linear relationship between runtime and the number of cells, indicating that these steps have algorithmic complexity higher than $O(n)$.

In contrast, other steps showed approximately linear scaling, which is desirable for handling larger datasets.

In this work, we will not investigate the optimisation of *divideByReference* and *impute* steps further. Indeed, the *divideByReference* step is not recommended in more recent SCP pipelines, and *impute* is not part of the *QFeatures* ecosystem as it relies on an external package. Moreover, imputation is generally not recommended for SCP data, reducing its practical relevance for optimisation in this context.

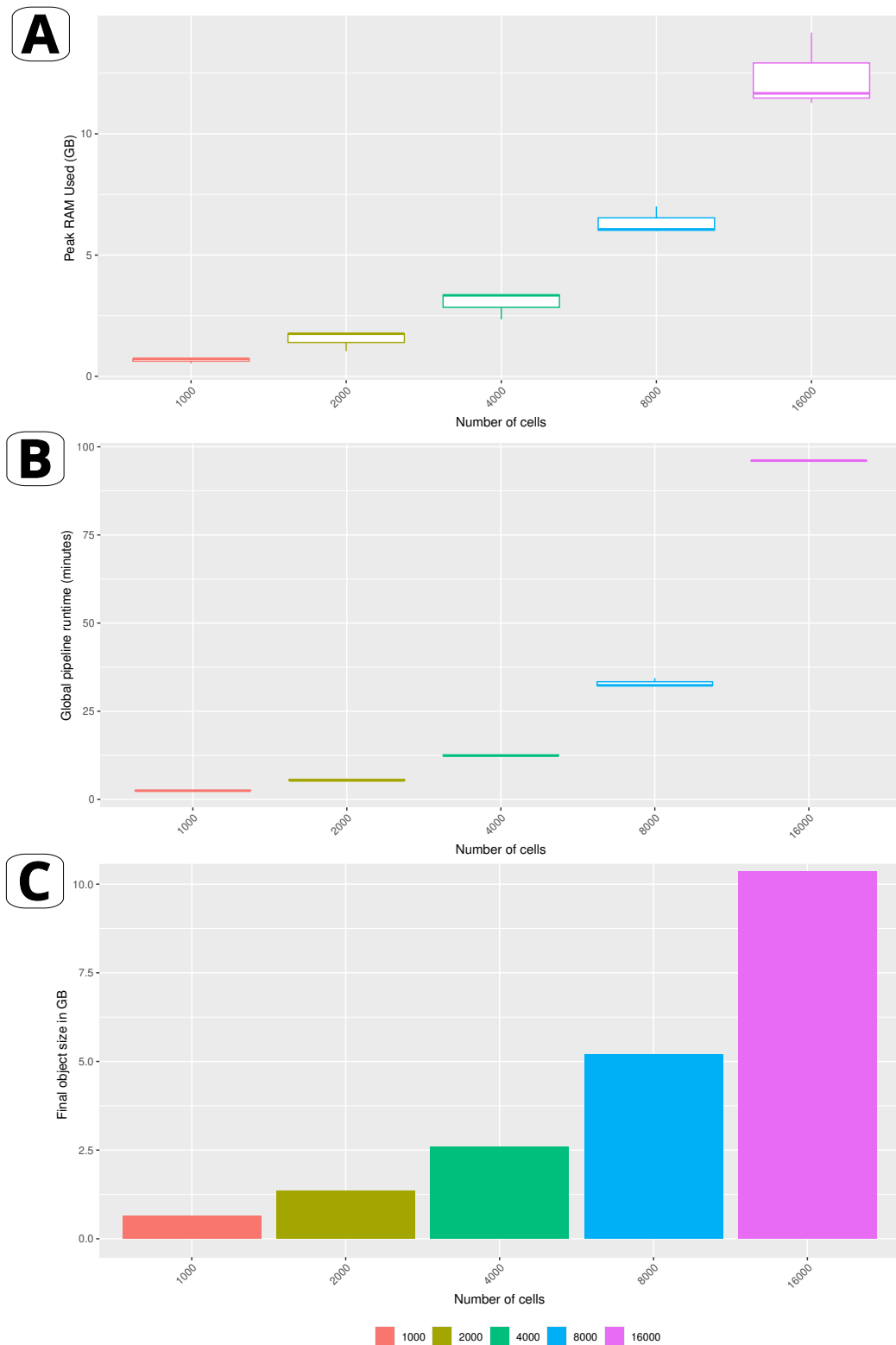


Figure 7. Leduc et al. benchmark results: metrics measured on the total pipeline, using different dataset sizes (1000, 2000, 4000, 8000, 16000) (x axis). **A:** Boxplot of the maximum of memory consumption on top of the in-memory object (peakRAM) in GB, for each dataset size we replicated the measure 3 times. **B:** Boxplot of the total execution time of the pipeline in minutes, for each dataset size we replicated the measure 3 times. **C:** Barplot of the size of the *QFeatures* object at the end of the pipeline in GB. Note that this measure is deterministic and doesn't vary between technical replicates.

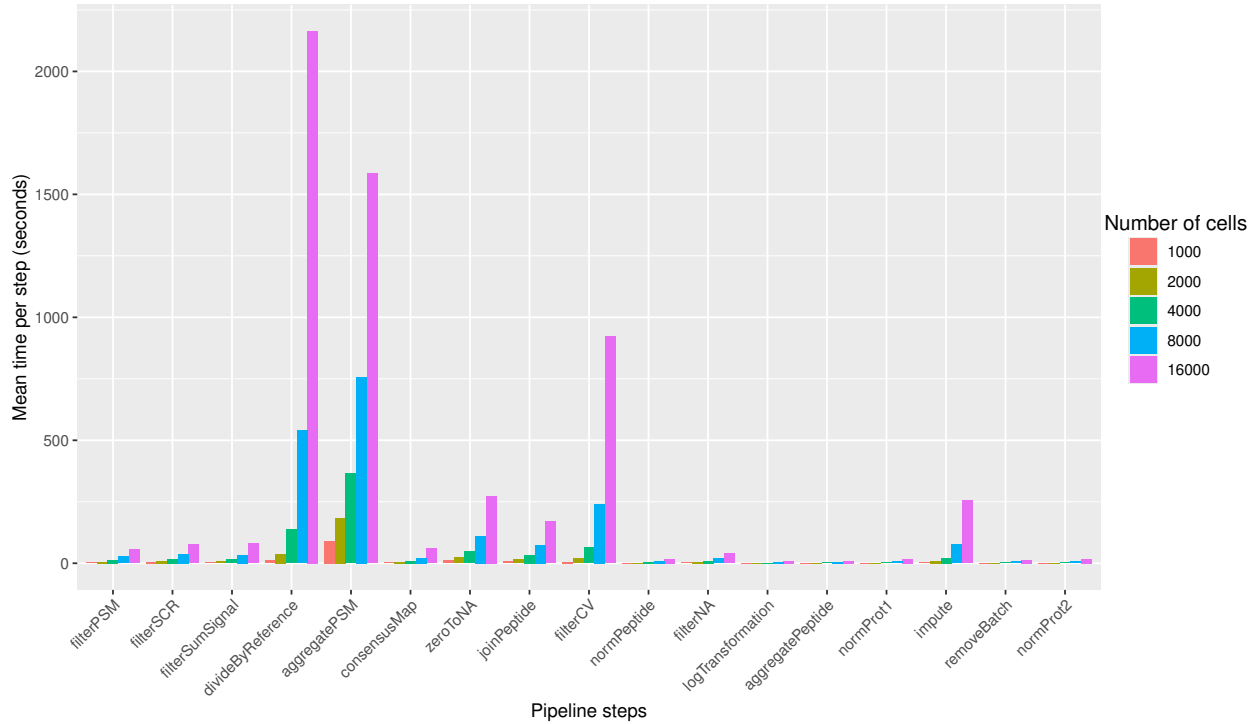


Figure 8. Leduc et al. benchmark results: per step runtime, using different dataset sizes (1000, 2000, 4000, 8000, 16000). For each dataset size (color) and step (x axis), the mean execution time in seconds is showed.

1.2. Individual Components Benchmark

The goal of this benchmark was to move from the conceptually complex, multi-step pipeline Leduc et al. benchmark to a more focused evaluation of the individual *QFeatures* and *scp* functions. By isolating and benchmarking single functions, we aimed to identify specific operations that drive overall pipeline cost in terms of runtime and memory usage.

Examining the global results (Figure 9), we see that three functions clearly exhibit higher runtimes than others: *aggregateFeaturesOverAssays*, *joinAssays*, and *subsetByColData*. These functions are widely used in single-cell proteomics (SCP) analysis pipelines, making them major computational bottlenecks that can limit scalability.

We next examined these three functions in detail, analysing both runtime and peak RAM usage (Figure 10). When looking at runtime, *aggregateFeaturesOverAssays* shows the highest runtimes across all dataset sizes (Figure 10 A). However, its runtime increases linearly with the number of cells, suggesting an algorithmic complexity close to $O(n)$. A similar linear trend is observed for *joinAssays* (Figure 10 C). In contrast, *subsetByColData* displays a clear non-linear relationship with input size (Figure 10 B), indicative of a complexity closer to $O(n^2)$. This suggests that subsetting becomes disproportionately expensive on larger datasets, which represent a major scalability concern.

For peak RAM usage (Figure 10 D–F), all three functions show an approximately linear relationship with the number of cells. Interestingly, although *aggregateFeaturesOverAssays* was the most time-consuming step, it is not the most memory intensive in terms of overhead. Instead, *joinAssays* has the highest peak RAM consumption. This can be explained by the fact that joining operations involve simultaneously combining all sets into a single set, effectively requiring an extra full in-memory copy of all sets. By contrast, *aggregateFeaturesOverAssays* operates set by set, sequentially, which limits RAM overhead.

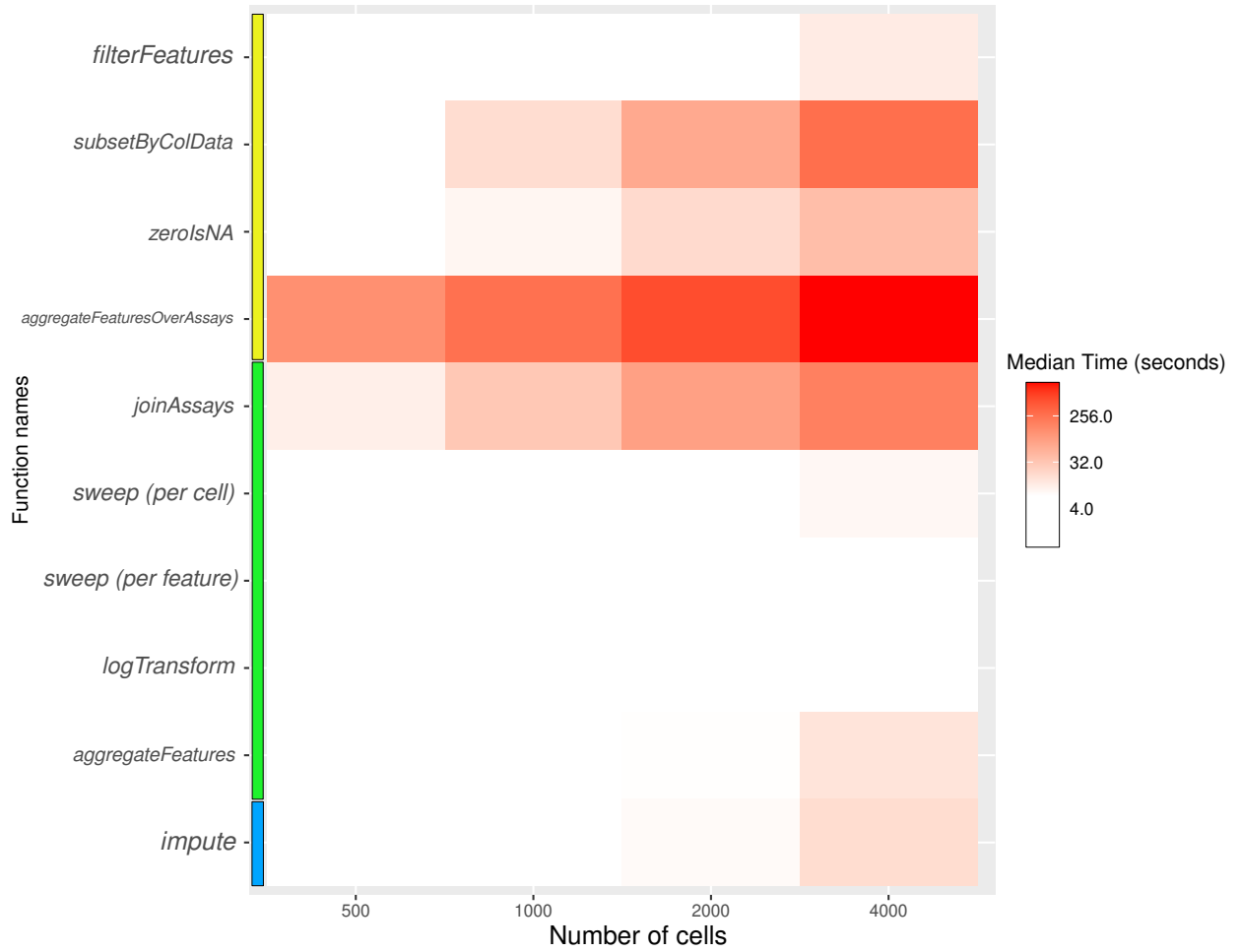


Figure 9. Heatmap of the benchmark of individual *QFeatures* and *scp* functions using different dataset sizes (500, 1000, 2000, 4000). Columns represents the number of cells per dataset. Rows represent each function benchmarked (as defined in the 2.2 Materials and Methods section). The color intensity represents then median execution time (more red equals more seconds). The color intensity use a log2 scale.

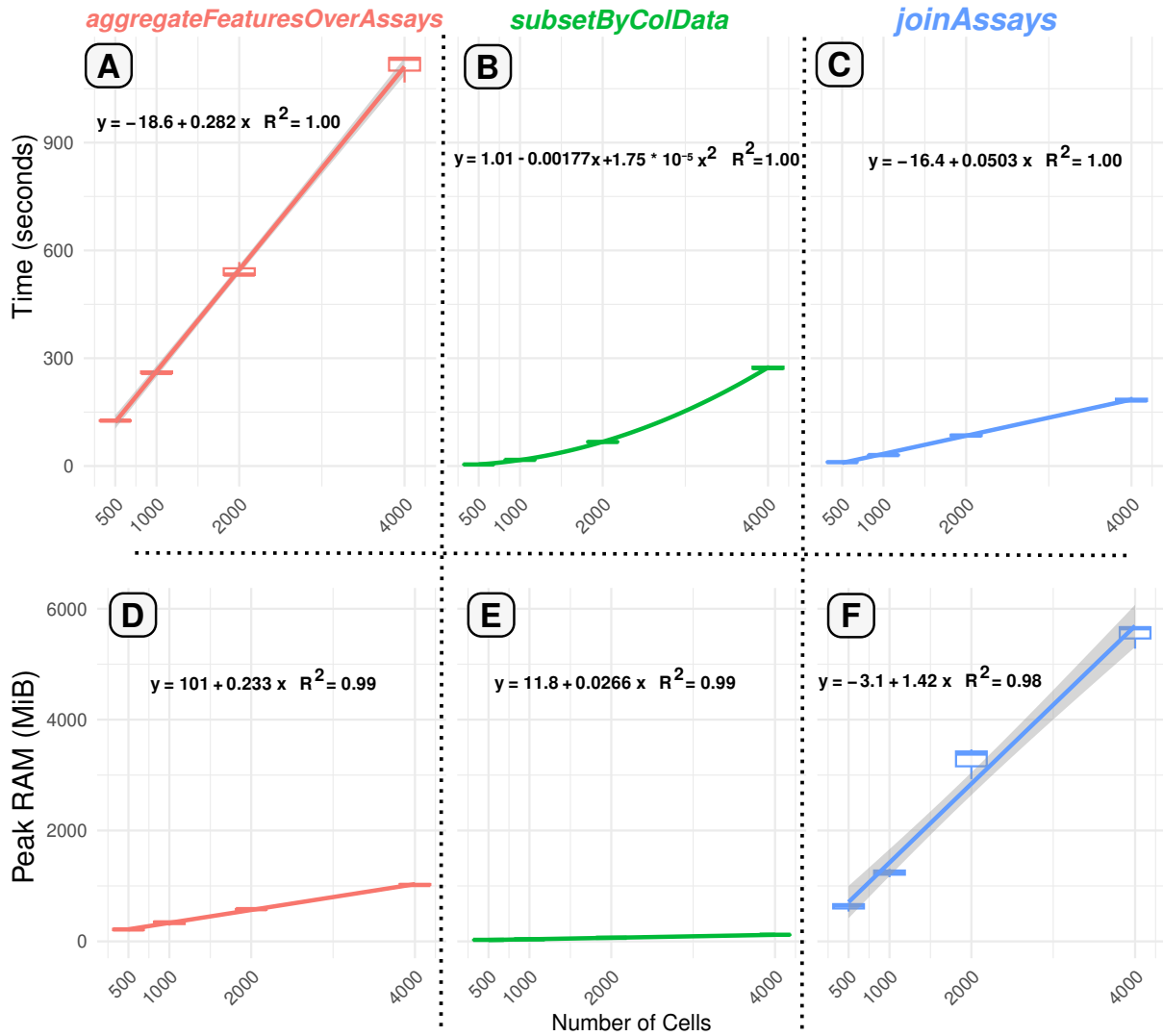


Figure 10. Boxplots of individual analysis of the `aggregateFeaturesOverAssays`, `subsetByColData` and `joinAssays` functions. The number of cells used as input is present on the y-axis. **A-C:** Execution time in seconds. **D-F:** Maximum of memory used during execution. For each plot, we model the relationship with a linear regression with one polynomial degree for **A, C-F** and two degrees for **B**.

1.3. QFeatures Object Memory Usage Benchmark

In this section, we present a benchmark analysis of how different variables impact the memory size of a `QFeatures` object. Figure 11 summarises the results. Four parameters were systematically varied: the number of sets, the number of cells per set, the number of features per cell, and the number of variables in the `rowData`.

A key insight from Figure 11 A is that the most relevant information is not in the absolute size values, but in the **slope** of the plotted relationships. Each column in the figure shows how the total object size (in MiB) varies with one parameter, while holding others fixed or iterating over different values. A steep slope indicates that increasing this parameter has a strong effect on object size, whereas a flat (or nearly horizontal) slope reveals minimal impact. For example, the plots corresponding to the number of cells per set show almost flat curves, indicating that this parameter has only a limited effect on total memory consumption.

Among the variables studied, the **number of sets** has the largest impact on the total size of the `QFeatures` object. This is because each set contributes its own quantitative matrix, row-level metadata (`rowData`), and sample-level metadata (`colData`). Adding sets increases all three slots

proportionally, resulting in a strong linear growth of memory usage, as reflected in the steep slopes in the top-left panel of Figure 11A.

The **number of rowData variables** ranks second in impact. As the row-level metadata accumulates, the *rowData* slot becomes increasingly dominant in the object's memory footprint. This is clearly illustrated in both Figure 11A and B, where the size of the *rowData* component increases substantially with more metadata columns.

The **number of features per cell** has a moderate effect, ranking third. It affects both the size of the assay (quantitative matrix) and the row-level metadata. Accordingly, its slope is positive and linear, but not as steep as that observed for the number of sets or row metadata columns.

In contrast, the **number of cells per set** has the smallest impact. Although more cells increase the number of columns in the assay matrix and entries in the *colData*, the associated memory increase is relatively minor. The flat slopes in the relevant panel confirm this limited effect.

Figure 11B further illustrates this by showing how slot-wise memory usage varies when increasing the number of cells per set while keeping the total number of cells fixed (by reducing the number of sets accordingly). The reduction in total memory consumption highlights a key design feature of *QFeatures*: it is more memory-efficient when handling multiplexed data with more cells per set and fewer total sets. This is primarily due to the reduction in duplicated features metadata in the *rowData* slot, which would otherwise be repeated across sets.

1.4. Challenging Dataset Benchmark

In this benchmark, we assess the memory and performance impact of optimizing the *rowData* content in a real world dataset comprising HeLa acquisitions. As shown in Figure 12 A, even when working with a subset of only 500 acquisitions out of the full set of 7,444, the imported *QFeatures* object occupies nearly 15 GB of memory at load time. This size further increases beyond 15 GB following the analysis steps, due to the creation of new data structures during aggregation and joining operations.

A key Optimisation was performed by subsetting the *rowData* slot to retain only the variables required for filtering and aggregation, reducing the number of variables from 61 to just 6. This subsetting leads to a threefold size reduction of the *QFeatures* object, from nearly 15 GB down to approximately 5 GB.

Beyond size reduction, this optimisation also translates into significant improvements in computational performance across several analysis steps (Figure 12 B). Most notably, the aggregation step exhibits a fivefold reduction in execution time, while the joining step is executed twice as fast. These improvements are particularly important since both steps were previously identified as bottlenecks in earlier benchmarks.

These results clearly highlight the importance of data pruning prior to analysis. Removing unnecessary metadata from the *QFeatures* object not only improves memory efficiency but also significantly accelerates processing, making this a crucial practice for large scale proteomics workflows.

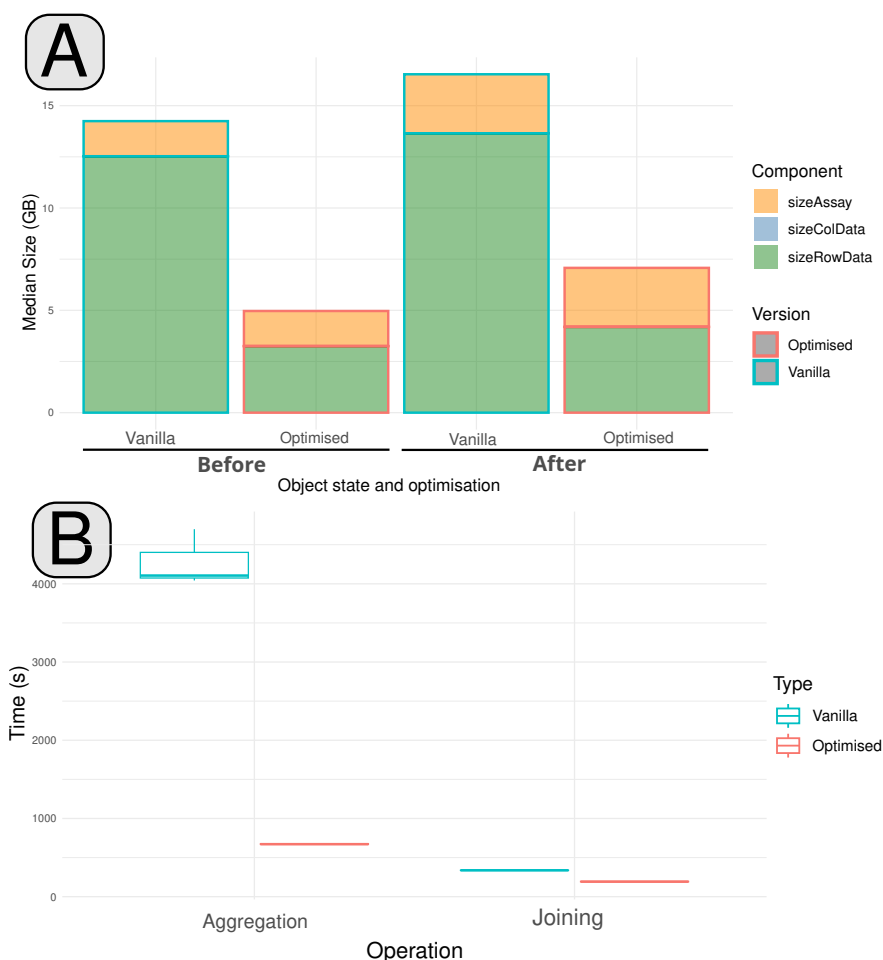


Figure 12. Benchmark results showing the effect of subsetting the `rowData` slot in a HeLa dataset. **A:** Object size (in GB) before and after the data analysis. Contribution of each slot to the global size is shown using yellow (*assay*), green (*rowData*) and blue (*colData*). The two different version, with (optimised) or without (vanilla) the subsetting of the *rowData* is highlighted respectively by a red and cyan frame. **B:** Boxplot of the execution time (seconds) for aggregation and joining steps between vanilla and optimized (*rowData* subsetting) versions.

2. Optimisations

2.1. subsetByColData Optimisation

The `subsetByColData` function was identified as the most critical bottleneck among all performance issues observed (cf. Results section 1.2. Individual Components Benchmark). Indeed, benchmarking revealed that this function exhibited a quadratic execution time trend. The function is implemented in the `MultiAssayExperiment` class, which is widely used, ranking 77th among 2341 Bioconductor packages. And it is used both on its own and as a dependency for other packages such as `QFeatures`, making this performance issue particularly concerning.

The `subsetByColData` function is employed whenever a user subsets the columns of the different experiments contained in a `MultiAssayExperiment` object for example, using the bracket subsetting syntax `object[, condition, ]`. Therefore, addressing its performance was crucial.

Using profiling tools to investigate the internal code, we found that the existing implementation performed unnecessary iterations. Specifically, for each set, it iterated over all column names of all sets, even though it only needed to consider the column names of the current set. A simple optimisation was introduced by restricting the iteration to the intersection of the retained columns (based on the subsetting condition) and the columns of the current set. This change reduced the complexity from $O(k^2)$ to $O(k)$, where k is the number of sets.

This optimisation resulted in significant improvements in both execution time and memory consumption, as illustrated in Figure 13. The fix was proposed in a detailed pull request on the MultiAssayExperiment GitHub repository⁴. It was reviewed, merged into the main branch, and is now available in the stable Bioconductor release (3.21).

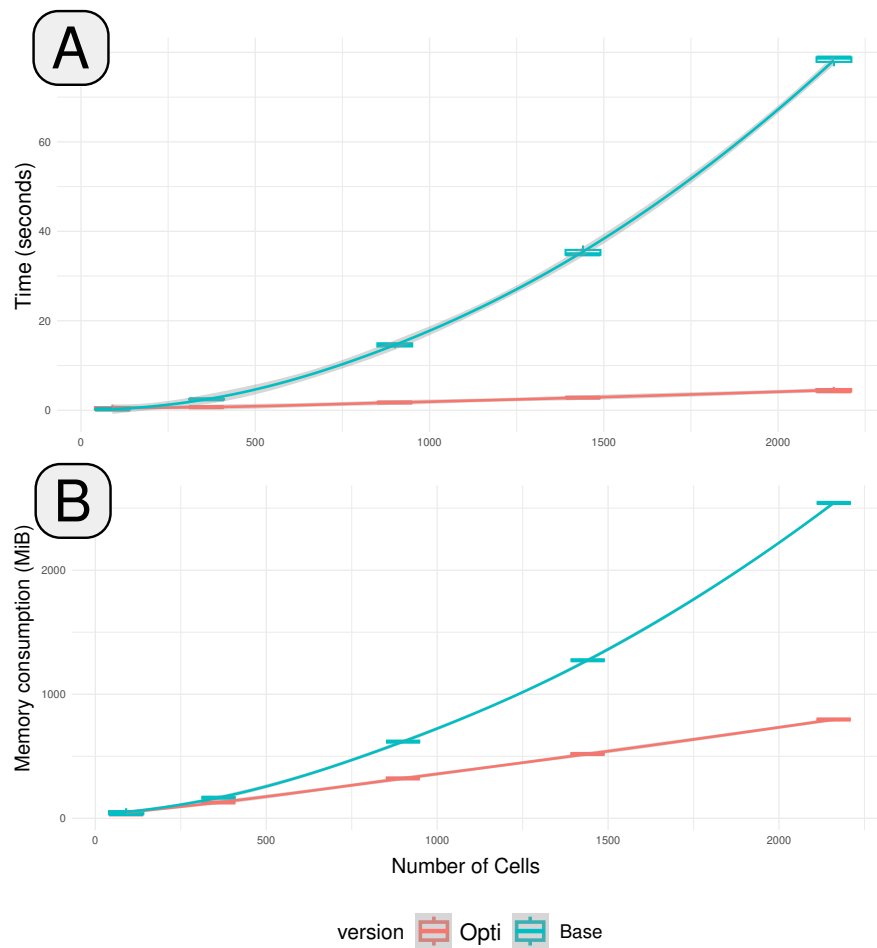


Figure 13. Boxplot of execution time in seconds (A) and memory consumption in MiB (B) for `subsetByColData` before (Base) and after (Opti) optimisation. The x-axis represent the input size as number of cells.

2.2. Aggregation Optimisation

The aggregation step was identified as the most time consuming operation in a typical SCP pipeline. Although its runtime did not show a nonlinear relationship with input size, we examined the `aggregateFeatures` function in detail to find ways to optimise it. Profiling and additional benchmarks revealed that the most expensive task during aggregation was the detection of "invariant" columns.

When aggregating several features into a new one, their associated metadata (*rowData*) must also be aggregated. For each metadata variable, the aggregation process checks whether all aggregated features from a group share a unique value, keeping it only if invariant, otherwise discarding it. This check is costly because it must evaluate each individual *rowData* variable for every group of aggregated features.

Originally, invariant columns were detected independently for each set. We introduced an optimisation trick by tracking all variables already flagged as variant, skipping their invariance check in subsequent sets. At the end, only variables invariant across all sets were retained. This approach leverages computations already performed, avoiding redundant work.

⁴<https://github.com/waldronlab/MultiAssayExperiment/pull/334>

We tested this optimisation across a range of input sizes and observed substantial performance improvements, particularly in scalability (Figure 14). Linear regression analysis revealed that the execution time of the original (base) implementation increased with input size with a slope of 0.27, whereas the optimised version exhibited a much flatter slope of 0.065, corresponding to a 75% reduction in growth rate. Similarly, memory consumption scaled more moderately in the optimised version, with a slope of 0.099 compared to 0.159 in the base implementation, representing an improvement of approximately 40% in growth rate. These results highlight that while both versions perform similarly on small datasets, the optimised implementation handles larger datasets much more efficiently. This optimisation has been reviewed and merged into the *QFeatures* package. It is currently available in the Bioconductor development version and will be included in the next stable release (3.22).

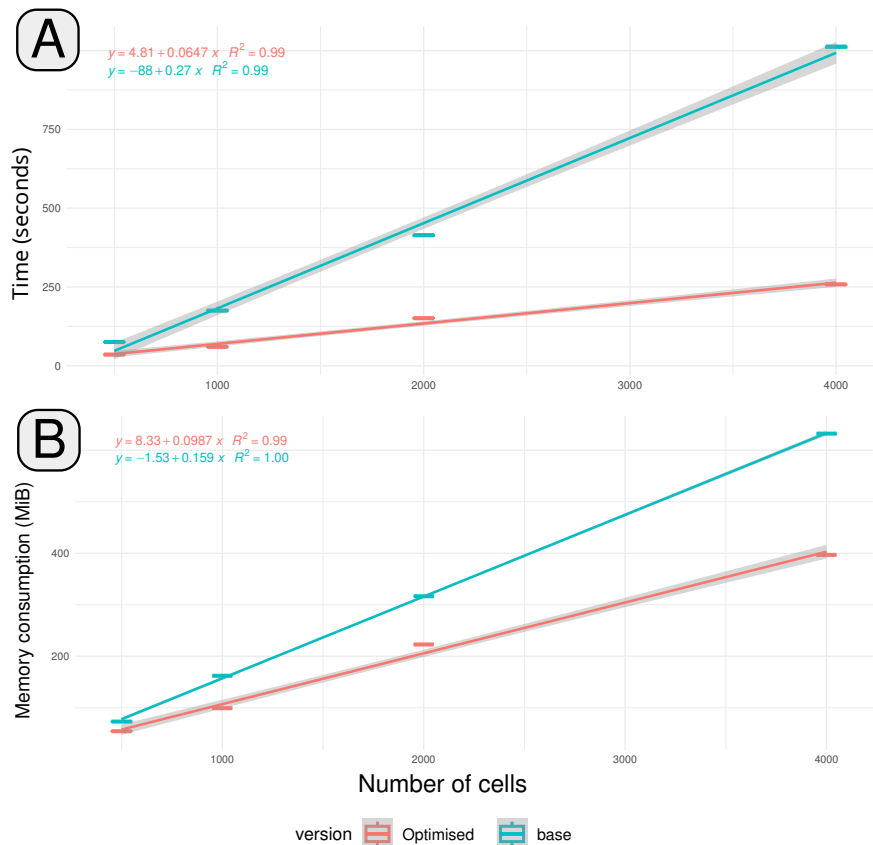


Figure 14. Boxplot of execution time in seconds (**A**) and memory consumption in MiB (**B**) for aggregation before (base) and after (optimised) implementation of the invariance *rowData* columns tracking strategy. The x-axis represent the input size as number of cells. Linear regression equation is shown for the different curves.

2.3. Using *SummarizedExperiment*

During the different benchmarks we conducted, we used the *QFeatures* and *scp* constructors (*readQFeatures*, *readSCP*) extensively. We observed that the *readSCP* constructor was almost twice as slow as *readQFeatures*. This is because *readSCP* internally calls *readQFeatures* and then converts the *SummarizedExperiment* objects in the resulting *QFeatures* to *SingleCellExperiment* objects.

We tried to modify *readSCP* to directly import the sets as *SingleCellExperiment*, expecting to avoid the conversion cost. However, profiling showed that this approach was even slower. We found that the *SingleCellExperiment* class inherits key methods from the *RangedSummarizedExperiment* class (a subclass of *SummarizedExperiment*), whose method implementations involve additional operations for *SingleCellExperiment*. While each of these operations is individually

minor, their cumulative effect can significantly degrade performance when executed many times.

Following this discovery, we assessed the impact of using `SummarizedExperiment` instead of `SingleCellExperiment` on several tasks. We reused the benchmark from Leduc et al. (2022) and applied it to two different *QFeatures* objects: one containing `SingleCellExperiment` objects and the other containing `SummarizedExperiment` objects. We observed reductions in execution time for all benchmark steps, with some steps showing up to 70% faster execution (Figure 15). Notably, the `divideByReference` step, which was previously identified as one of the most time-consuming operations in the real pipeline benchmark, benefited substantially from the switch, showing a marked decrease in execution time. This highlights the practical relevance of the change in performance-critical sections of the workflow.

While `SingleCellExperiment` is the standard class for single-cell data, the `scp` infrastructure supports all functions with `SummarizedExperiment` objects as well. The only part of `scp` that requires `SingleCellExperiment` is *scplainer*, the modelling method that can be used to perform dimensionality reduction and that store the results in a dedicated `SingleCellExperiment` slots.

Given these findings, we decided to change the default import behaviour of the `readSCP` constructor to use `SummarizedExperiment` objects. All functions were adapted to work with both SE and SCE objects. The *scplainer* modelling tool was modified to accept and automatically convert SE objects if provided. The documentation across the `scp` package was updated to highlight this paradigm change. All these changes were reviewed, merged into `scp`, and are now available in the devel Bioconductor release, with plans to ship in the next stable release (3.22).

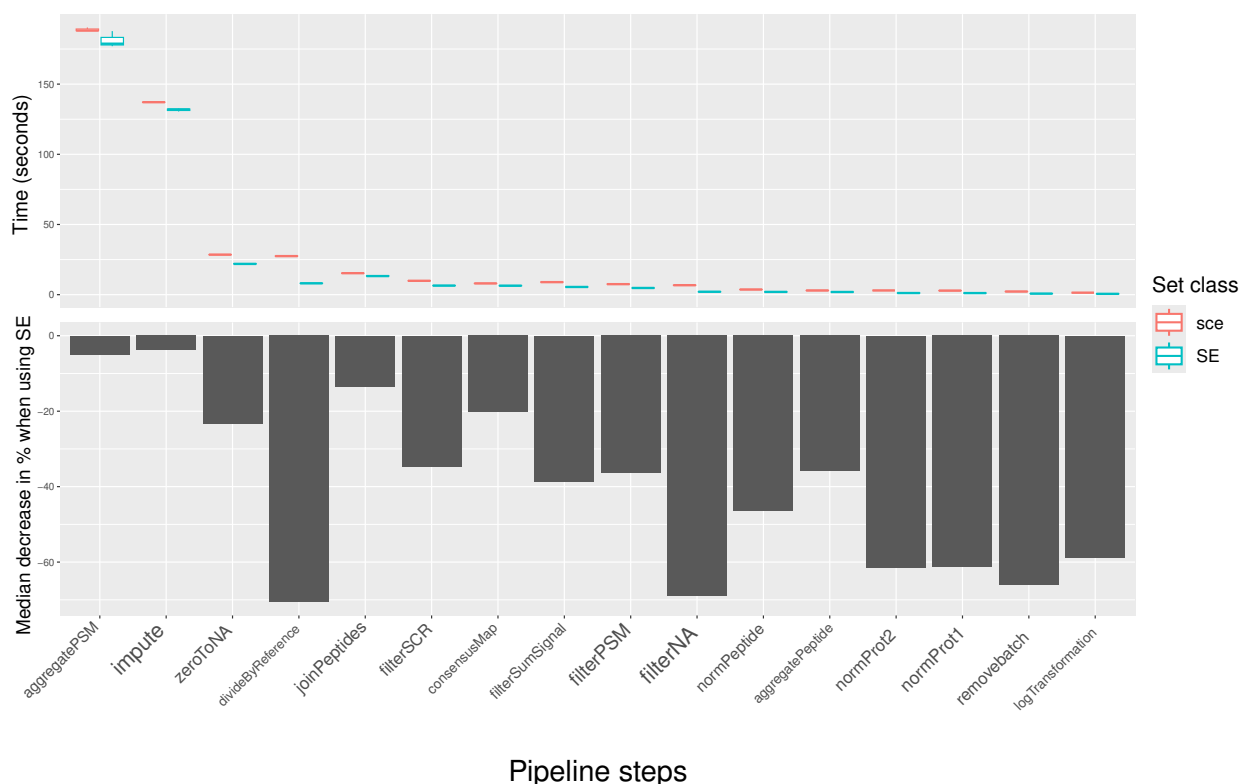


Figure 15. Per step boxplots of the execution time (in seconds) comparison between `SummarizedExperiment` (blue) and `SingleCellExperiment` (red) class usage in the *QFeatures* object. The per step decrease in % is shown below.

3. PseudoBulking

3.1 Simulation Quality Control

To ensure the robustness of our simulation framework, we performed a set of quality control assessments to verify the integrity and biological plausibility of the generated data.

We first focus on the experimental distribution of the three proteins with the highest number of non-missing values, to see if they can be approximated using normal distribution. As shown in Figure 16 A, two of these proteins, *CLIC1* and *POSXL*, exhibit log-intensity experimental distributions that closely resemble a normal distribution (red dashed line in the figure), suggesting that our simulation can effectively reproduce the expression patterns of these proteins. In contrast, the *NACAM* protein displays an experimental distribution with three distinct peaks, deviating from normality. This deviation is evident when comparing the experimental distribution to the fitted normal distribution derived from the same data: the normal curve is flatter and centered at a value that does not align with any peak from the experimental data.

Although our normal-distribution-centered simulation strategy does not perfectly capture such complex multi-modal patterns, it nevertheless serves as a reasonable and straightforward approximation of the experimental distributions.

We further evaluated whether the effects (cell type and patient) were successfully inserted into the simulated data. To do so, we employed the component analysis functionality of the *scplainer* method, which integrates linear modeling with an APCA+ (extended ANOVA-simultaneous component analysis) [67]. This method reconstructs the data attributed to each effect while retaining residual variation from unmodelled sources. It provides a powerful tool to visualise and validate the contribution of individual effects.

The APCA visualization for the cell type effect (Figure 16 B) reveals five distinct clusters of cells in the first two dimensions, each corresponding to one of the five simulated cell types. This confirms that the cell type effect has been correctly incorporated into the simulation.

Similarly, the APCA plot for the patient effect (Figure 16 C) shows 16 discernible clusters, matching the number of simulated patients. The varying degrees of separation between patient clusters can be attributed to the random nature of the shift generation. Since patient-specific shifts are sampled from normal distributions centered at zero, some patients may share similar shift values, resulting in partially overlapping clusters.

Overall, these quality control results demonstrate that the intended design has been faithfully reproduced in the simulated data.

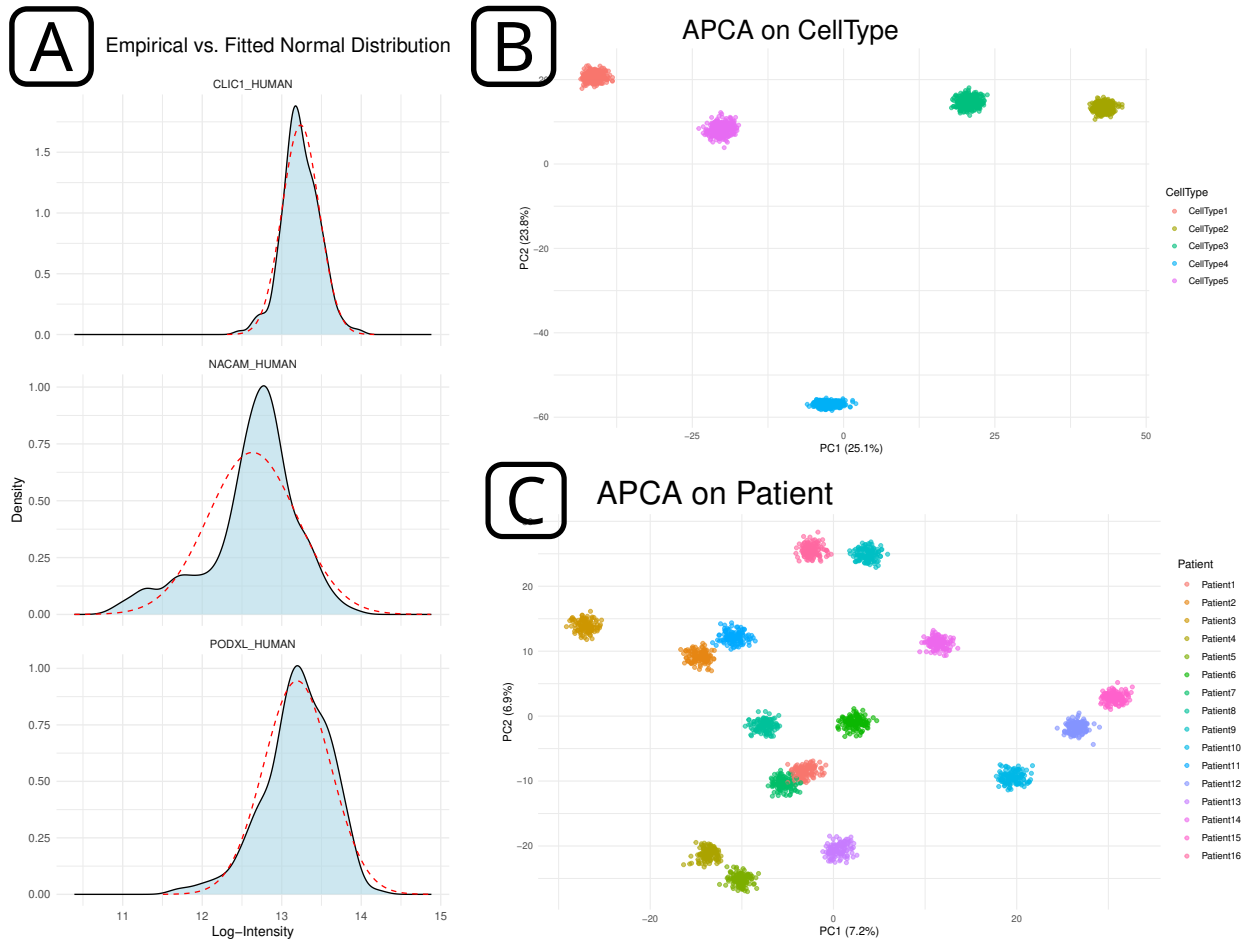


Figure 16. Simulation Quality Control. **(A)** Experimental distribution (blue area) of log-transformed protein intensities for the top 3 proteins with the less missing values, fitted normal distributions obtained from the same data are represented by red dashed lines. **(B)** APCA+ dimension reduction plot on the simulated cell type effect showing five distinct clusters (colors = cell type). **(C)** APCA+ dimension reduction plot on the simulated patient effect showing sixteen distinct clusters (colors = patient).

3.2 Impact of the shift value on performances

To evaluate how the strength of the simulated treatment effect influences model performance, we tested a set of shift values ranging from 0.1 to 0.25, scaled on the mean protein intensity (μ_i). Results are shown in Figure 17.

The shift value has a strong impact on performance. Larger shifts lead to higher true positive rates (TPR), while maintaining false discovery rate (FDR) control.

Analysing the performance across modelling strategies:

- As expected due to its incomplete model design, **scplainer** exhibits poor performance across all shift values. It fails to control the FDR, which reaches values as high as 60%.
- **Pseudobulking methods (mean, median, robustSummary)** perform consistently well, achieving strong TPR and maintaining low FDR across all shift values. Results from these three aggregation strategies are nearly identical. However, the control of the FDR is not perfect, as the observed FDR is consistently shifted relative to the adjusted p-value threshold. This discrepancy decreases as the shift value increases.
- **msqrob2** achieves slightly higher TPR values than the pseudobulk methods, but the difference is small and unlikely to be significant. The largest gap is observed at a shift value of 0.15, where msqrob2 reaches a TPR of 0.68 compared to 0.63 for pseudobulk (Table S1). This small gain in TPR is accompanied by a slight increase in FDR (from 0.07 for pseudobulk

to 0.08 for msqrob2 (Table S1)), suggesting no clear advantage. Overall, its performance is nearly indistinguishable from that of the pseudobulk methods. As with pseudobulking, the control of the FDR is not perfect, with a consistent shift between the adjusted p-value threshold and the true FDR, which diminishes as the shift value increases.

- **Sum aggregation**, in contrast, performs poorly with noticeably lower TPR and higher FDR.

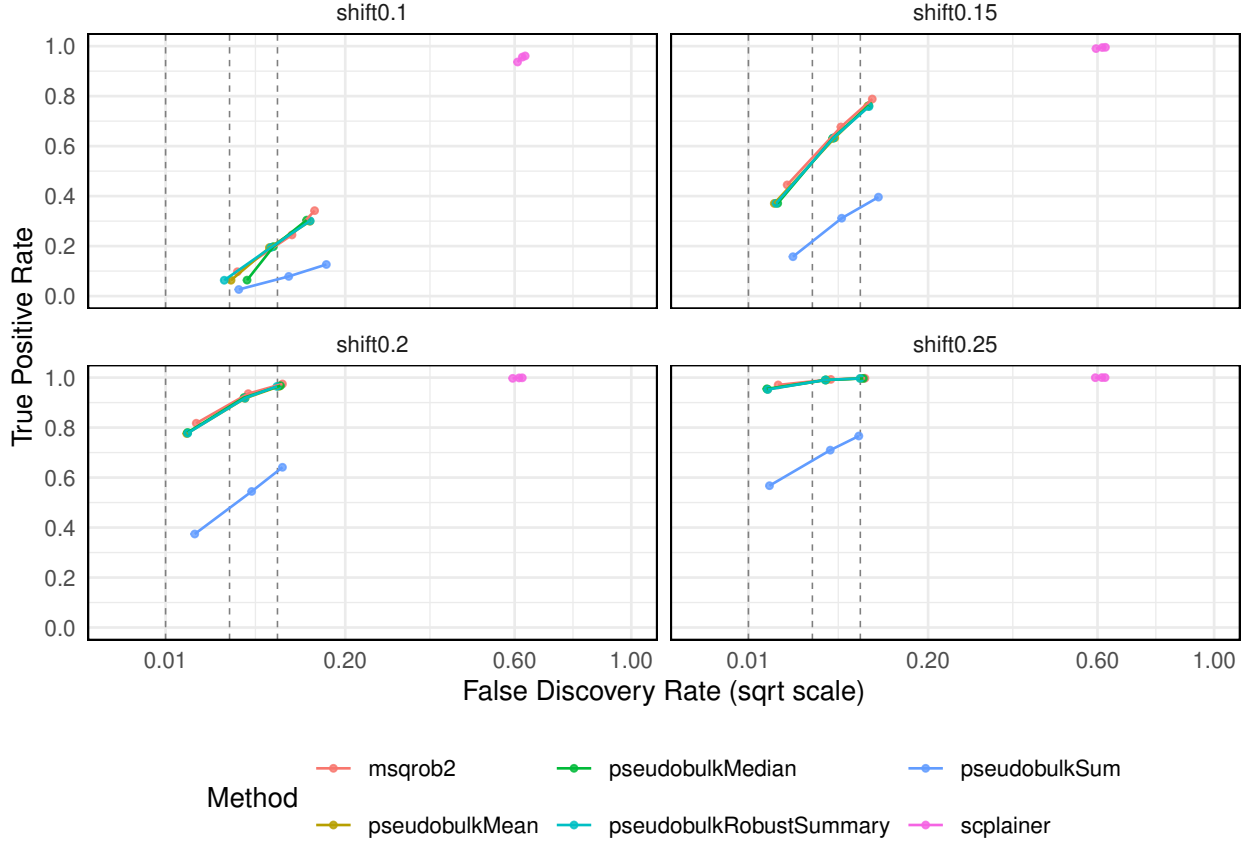


Figure 17. Mean True Positive Rate (TPR) vs. Mean False Discovery Rate (FDR) for different shift values, scaled to the mean protein intensity. Each subplot corresponds to a specific shift value. For each method, TPR and FDR are reported at three adjusted p-value thresholds: 0.01, 0.05, and 0.1. The FDR is displayed on a square root scale; dashed lines indicate FDR values of 0.01, 0.05, and 0.1. Colors represent different methods. Note that the mean FDR and TPR values were obtained with the 3 simulation replicate (using different seeds). Note that the number of cells per condition is fixed at 50.

3.3 Impact of the number of cells on performances

We next assessed how varying the number of cells per cell type per patient influences model performance (Figure 18). Here we focus on msqrob2 and pseudobulk models using mean, median, and robustSummary aggregation as other models (scplainer and sum aggregation show overall poor performances).

Overall, increasing the number of cells per group leads to moderate improvements in performance, with the largest change occurring between 10 and 25 cells. The lowest TPR and highest FDR occur at 10 cells per combination, confirming that a minimal number of observations can limit model sensitivity. For example, at the 0.05 adjusted p-value threshold, the TPR of msqrob2 increases from 0.63 at 10 cells to 0.67 at 25 cells, and then only slightly to 0.68 at both 50 and 100 cells (Table S2), indicating that differences between 25, 50, and 100 cells are negligible.

In conclusion, while increasing cell numbers per group from very low counts (10 cells) can improve performance, further increases offer little to no benefits.

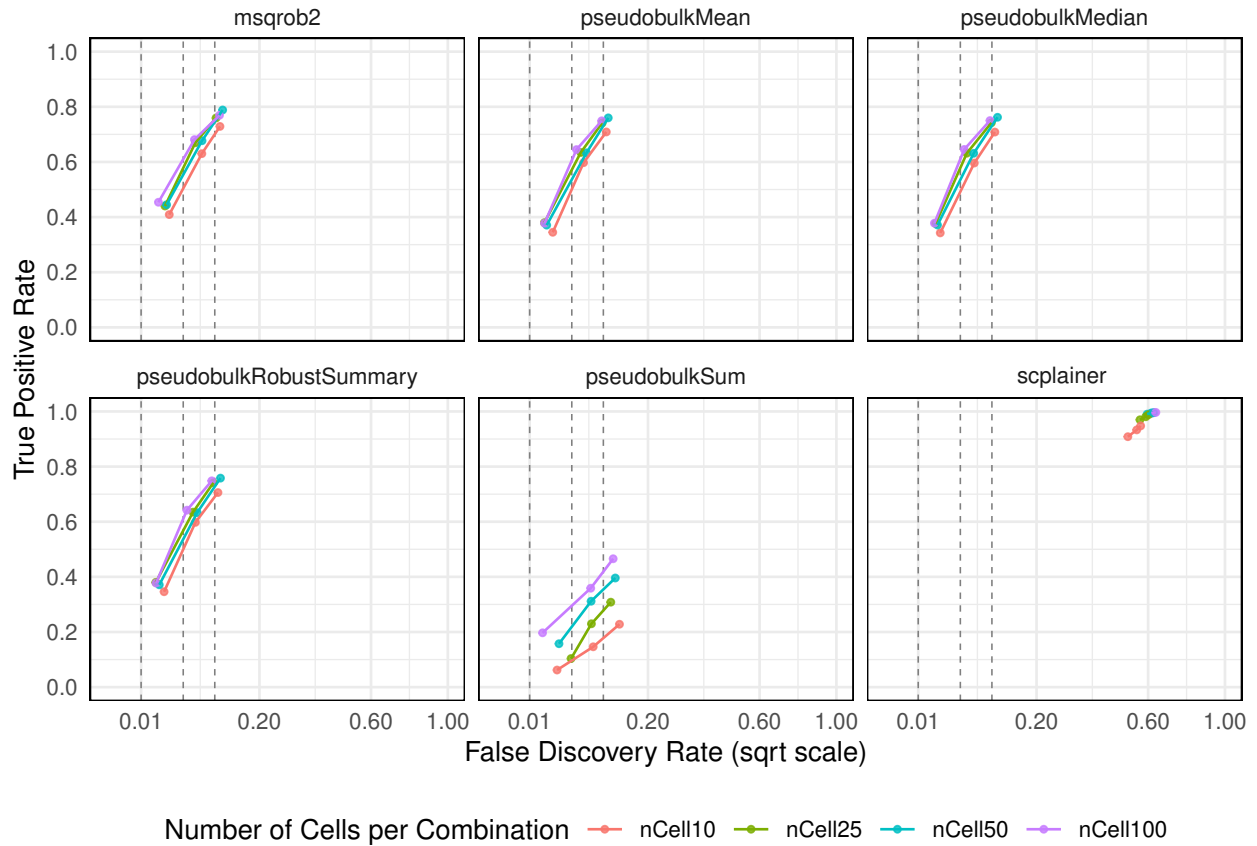


Figure 18. Mean True Positive Rate (TPR) vs. Mean False Discovery Rate (FDR) for different numbers of cells per condition (cell type/patient). Colors represent different cell counts. For each method, TPR and FDR are shown at three adjusted p-value thresholds: 0.01, 0.05, and 0.1. The FDR is plotted on a square root scale; dashed lines mark FDR values of 0.01, 0.05, and 0.1. Each subplot shows the performance of one method. Note that the mean FDR and TPR values were obtained with the 3 simulation replicate (using different seeds). Note that the treatment shift value is fixed at 0.15

3.4 Runtime analysis of the modelling methods

We compared the computational runtime of all modelling strategies as a function of dataset size (Figure 19).

- **msqrob2** scales linearly with data size ($O(n)$) and achieves faster runtimes primarily due to its use of parallelism (4 cores).
- **scplainer** is slightly slower than msqrob2 in its current implementation, which uses only a single core. It exhibits a sub-linear runtime trend ($< O(n)$, possibly $O(\log n)$), suggesting good scalability with increasing number of cells. The observed difference in runtime compared to msqrob2 is relatively small and could likely be eliminated by introducing parallelism, as msqrob2 currently benefits from using 4 cores.
- **Pseudobulk methods (mean, median, sum)** show near-constant runtime. This is because the aggregation reduces the number of data points to 80 (16 patients $\times$ 5 cell types), regardless of the original cell count. The aggregation step itself represent a fraction of the total runtime and can easily scale as the input size increase.
- **Pseudobulk with robustSummary** has the longest runtime. This is due to the model fitting of one robust model per protein and per group, which results in fitting thousands of small models (e.g., number of proteins $\times$ 80 groups). This implementation was not optimized for parallel execution, which could significantly reduce runtime in future work. Note that even if it is the slowest method it exhibits a sub-linear runtime trend.

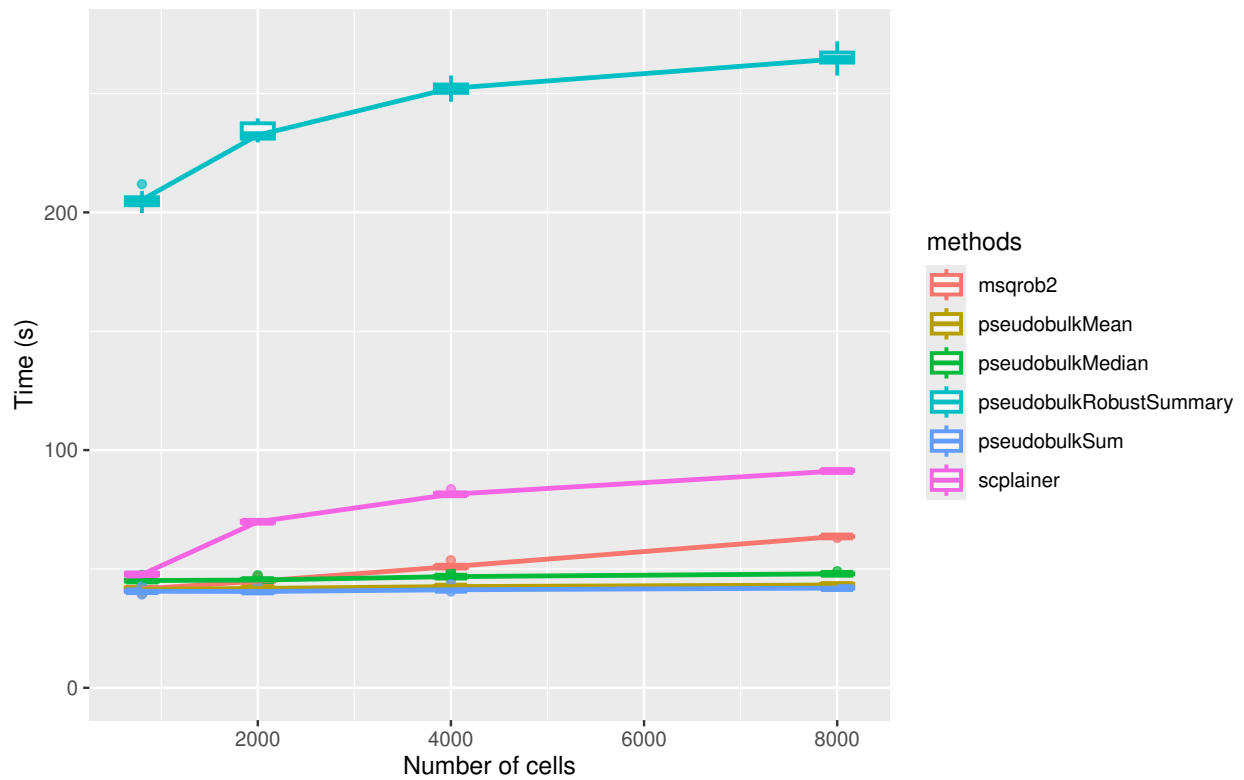


Figure 19. Boxplots of the runtime of the tested modelling methods. Measured runtime as a function of the number of simulated cells (800, 2000, 4000 and 8000 cells). There are 4 replications per method/number of cells. Colors represent the different modelling methods.

Discussion

1. Optimisations

1.1 Implemented Optimisations

In this work, we implemented different optimisations to improve the computational performance of the *QFeatures* and *scp* packages. These optimisations were target for discovered bottleneck functions (*aggregateFeatures* and *subsetByColData*). Furthermore, we also adapted the *scp* package to accept *SummarizedExperiment* instead of *SingleCellExperiment*, the first showing better performance over the second. We also suggest to subset the features metadata to only keep variable used in the analysis as this metadata can be very large.

To assess the impact of these changes, we benchmarked execution time and memory consumption using the *VerR* package (Figure 20). Three functions (*filterFeatures*, *subsetByColData*, and *aggregateFeatures*) were benchmarked, as they were expected to be most sensitive to these optimisations and are the most common and crucial steps in a SCP analysis.

Runtime Improvements. The most substantial performance gains were observed for *subsetByColData* and *aggregateFeatures* for which targeted optimisations were implemented, both functions exhibited significant reductions in execution time following the targeted changes. Also, switching from SCE to SE had a measurable effect on the aggregation step, likely due to reduced overhead associated with the *SummarizedExperiment* object methods. Similarly, pre-filtering the *rowData* further accelerated computations during aggregation. When combining all optimisations, the median total execution time dropped from 1457 seconds to just 59 seconds, showing a 96% reduction.

Memory Usage. In terms of memory consumption, optimising *aggregateFeatures* led to measurable savings for that specific function. The switch to SE, however, did not result in substantial memory improvements. In contrast, subsetting the *rowData* yielded a marked reduction of approximately 68%. Altogether, the combination of optimisations decreased overall memory usage by about 70% in our test setup.

These results underscore the critical importance of the implemented optimisations. Together, they deliver substantial improvements in both speed and memory usage, most notably a 96% reduction in execution time and 70% lower memory consumption, transforming *QFeatures* into a significantly more efficient framework. These enhancements directly address the growing demands of single-cell proteomics by enabling the analysis of increasingly large and complex datasets without compromising responsiveness or usability. Importantly, all code introducing these optimisations underwent thorough code review and passed comprehensive unit tests to ensure correctness and maintain high-quality standards. The optimisations are already integrated into the Bioconductor development version and will be part of the upcoming stable release (3.22), ensuring that users can immediately benefit from these improvements in future workflows.

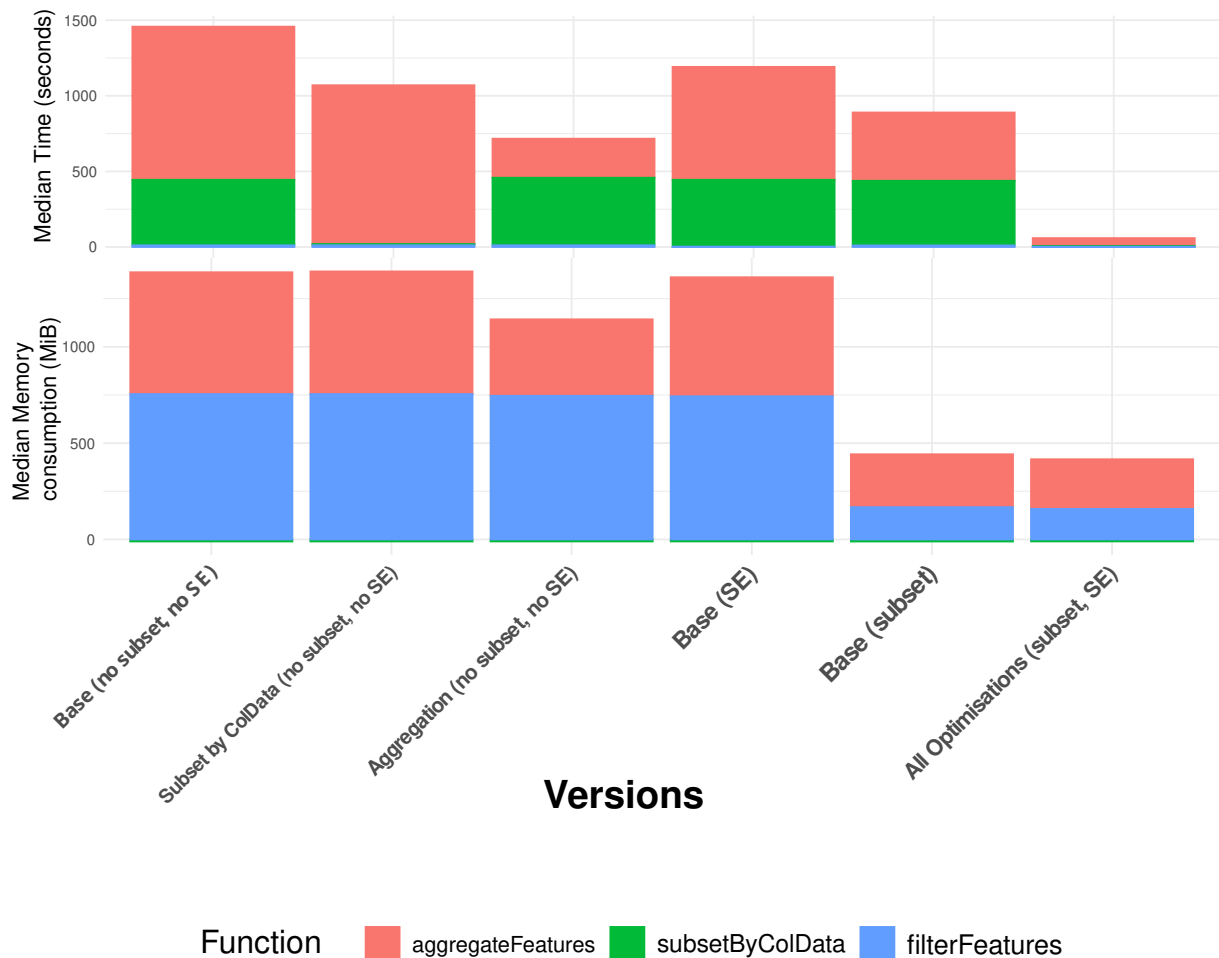


Figure 20. Barplot of execution time (in seconds) and memory consumption (in MiB) across different optimisation strategies. Colours indicate the contribution of each function call to the total runtime and memory usage.

1.2 Looking Ahead: Potential Next Optimisations

While the optimisations described above provide substantial improvements in performance, additional high-level strategies can be explored to further enhance the QFeatures and scp frameworks, particularly in anticipation of increasingly large and complex datasets in single-cell proteomics.

Parallelism. Many operations in QFeatures iterate over experiments independently, applying the same logic to each. For example, methods such as `aggregateFeatures` or `impute` perform computations on each set without interdependence, making them amenable to parallel execution.

The `BiocParallel` package provides a simple and consistent interface for parallel computing in R. Integrating it into the QFeatures framework would enable the distribution of experiment-level operations across multiple CPU cores. This could result in near-linear reductions in runtime for many performance-critical functions, with relatively limited code refactoring required.

Strategies for Reducing Memory Consumption. Despite these performance gains, memory usage remains a major limiting factor, particularly evident when analysing large datasets such as the HeLa QFeatures object, which can require over 200 GB of RAM. To address this challenge more systematically, we outline two complementary strategies:

- **Sparse Arrays with NA Support.** SCP datasets typically contain a high proportion of missing values (often over 85%), which are encoded as NAs. Conventional sparse matrix

formats in R assume zero as the default value, making them unsuitable for SCP data where zeros and NAs carry distinct semantic meanings.

A promising solution is the experimental `NaArray` class from the `SparseArray` package [68], which allows NA to serve as the sparse default. Incorporating this data structure into the quantitative data layer of `QFeatures` could yield substantial memory savings without compromising the correctness of downstream analyses. Preliminary benchmarks using the Leduc et al. (2022) dataset show a 50% reduction in quantitative data size when using `NaArray`, demonstrating the strong potential of this approach.

However, sparse arrays come with trade-offs between memory efficiency and runtime performance. Some operations require more computational steps on sparse matrices, this is explained by the added complexity to access the values contained in the matrices. Also if a needed operation is not implemented for sparse arrays, the object must first be converted to a dense matrix (classical matrix). This conversion increases execution time and temporarily doubles memory usage, as an in-memory dense copy of the matrix is required. Moreover, integrating `NaArray` into `QFeatures` would add complexity to the codebase, potentially making it harder to debug and extend in the future.

- **On-Disk Data Representation.** A more scalable, long-term approach involves decoupling data storage from memory by leveraging on-disk data representations for both quantitative data and metadata:
 - Quantitative data (dense numeric matrices) could be stored using disk-backed formats such as HDF5 (Hierarchical Data Format, version 5) [69, 70], which support partial loading and block-wise access.
 - Metadata stored in the `rowData` slot—typically composed of heterogeneous data types—could benefit from columnar formats such as Apache Parquet, supported via the `arrow` R implementation [71], enabling selective column access and efficient on-disk querying.

However, this architectural shift introduces a fundamental trade-off: reducing memory usage via on-disk storage often comes at the cost of slower runtime performance. Accessing data from disk is significantly slower than in-memory operations, which can negatively impact the responsiveness of the software, particularly in interactive settings where users expect fast feedback. Thus, adopting on-disk representations requires careful consideration of usage context: while ideal for batch or high-throughput processing of large datasets, they may degrade the user experience for smaller, exploratory workflows.

Moreover, implementing such on-disk infrastructure would require substantial internal refactoring of the `QFeatures` package, including extending `arrow` package and adapting key methods to support lazy loading and block-wise computation.

Together, these memory-saving strategies, NA-aware sparse arrays and disk-backed storage, offer promising directions to improve scalability. Yet, they must be weighed against their trade-offs in speed and implementation complexity, underscoring the need for flexible, context-aware solutions as SCP datasets continue to grow in size and complexity.

1.3 Beyond Performance: Knowing When to Stop

Optimising software performance is an essential part of building usable and scalable tools, especially in computational biology, where datasets can be large and complex. However, performance optimisation must ultimately serve a purpose: aligning the capabilities of the software with the needs of its users.

In the context of single-cell proteomics (SCP), processing speed should not become a bottleneck that discourages or delays interactive data analysis. Users need to be able to quickly filter, aggregate, and visualise their data to make informed decisions. Since R remains one of the most effective environments for interactive and exploratory data analysis, it is critical that tools like *QFeatures* and *scp* maintain responsiveness and efficiency during typical usage scenarios.

That said, optimisation is not a goal in itself, it involves trade-offs.

The Cost of Premature Optimisation. In the early stages of software development or when designing new features, focusing too heavily on optimisation can be counterproductive. Time spent micro-optimising code may delay the delivery of core functionality or reduce flexibility when requirements change. As famously noted by Donald Knuth, "Premature optimisation is the root of all evil." [72]. Performance should be addressed when it is proven to be a limiting factor, not in anticipation of hypothetical bottlenecks.

Complexity vs. Maintainability. High-performance implementations often come with increased architectural complexity. For example, introducing an on-disk data management system would undoubtedly solve the memory usage issue, but it would also add multiple layers of abstraction, increase the likelihood of bugs emerging, and make the codebase harder to maintain or extend. Each new optimisation, especially those that touch core data structures, needs to be weighed against its long-term impact on the maintainability and clarity of the software.

Sustainable Optimisation. A sustainable approach to performance engineering involves:

- Prioritising bottlenecks identified through profiling and real-world usage.
- Leveraging mature, external libraries (e.g., *arrow*, *SparseArray*, *BiocParallel*) rather than reinventing lower-level infrastructure.
- Ensuring that performance gains do not compromise reproducibility, transparency, or user experience.

Optimisation should be viewed as a means to an end, not an end in itself. Once the software performs sufficiently well to meet user needs in realistic scenarios, such that performance is no longer a primary constraint, future development efforts may be more productively directed toward improving usability, documentation, extensibility, and overall functionality.

2. Pseudobulking

2.1 Insights from Simulation Studies

Using our simulation framework, we evaluated the performance of various modelling approaches for differential analysis in single-cell proteomics (cf. Results section 3). Several important conclusions emerged:

Inadequacy of *scplainer*. The *scplainer* method fails to control the false discovery rate (FDR). This limitation stems from its inability to model the patient effect as a random factor. In our setup, the treatment is applied to half of the patients. This makes the addition of both patient and treatment effect in our linear model impossible. As a result, the patient variability is not explicitly modelled, the method will therefore incorrectly attributes it to the treatment effect, resulting in an inflated number of false positives and an overestimated treatment effect.

***msqrob2* and *pseudobulk* perform similarly.** Across all configurations, the *msqrob2* method and *pseudobulk* approaches (mean, median, *robustSummary*) exhibited very similar performance in terms of FDR control and detection power. While *msqrob2* achieved slightly higher TPR values

in some settings, these differences were small and unlikely to be significant (e.g., at a shift value of 0.15, TPR of 0.68 for `msqrob2` vs. 0.63 for pseudobulk), and were accompanied by a marginal increase in FDR (from 0.07 to 0.08). Given these minor differences, both strategies can be considered equally effective for multi-patient multi-condition SCP datasets. This also suggests that important signal is retained after aggregation, and that effective modelling is possible without individual cell-level resolution.

Limited sensitivity to number of cells. The number of cells used for the simulations had limited influence on models performances. Using as few as 10 cells per condition still produced satisfactory results, supporting the feasibility of pseudobulk methods in practical SCP settings, even when cell numbers per condition are low. Increasing the number of cells per condition above 25 did not lead to any significant performance gains, with results for 25, 50, and 100 cells being nearly identical.

Sum aggregation fails. Aggregation by sum yielded poor results across the board. This stems from SCP data's high proportion of missing values. Summing implicitly treats missing values as zeros, which biases the aggregated intensities and distorts the underlying signal. In contrast, methods like mean or median are more robust to missingness.

Pseudobulk methods are faster. In terms of runtime, pseudobulk approaches are markedly faster than modeling non-aggregated data with `msqrob2`. Since pseudobulking reduces the number of observations, the modeling step becomes computationally lighter. This speed advantage grows with increasing cell numbers, as the pseudobulk runtime remains nearly constant. Note that our aggregation implementation is experimental and could be optimised (e.g., via parallel processing).

Poor runtime performance of `robustSummary` aggregation. The `robustSummary` aggregation showed worse runtime than both mean and median in our benchmarks. Since the model performance obtained with this method is similar to mean and median methods and its higher computational cost, we do not recommend to use *robustSummary* in this context.

Recommendation. Given the very similar performance of `msqrob2` and pseudobulk approaches (mean, median, `robustSummary`) in terms of FDR control and detection power, either strategy can be effectively applied to multi-patient multi-condition SCP datasets. While `msqrob2` offers a flexible modelling framework for both fixed and random effects, pseudobulk methods provide comparable statistical performance with the added advantage of reduced computational complexity, making them an equally valid choice, particularly for large datasets or when runtime is a limiting factor.

2.2 Simulation Limitations

Although the simulation framework presented in this study provides a valuable first exploration of pseudobulk methods for single-cell proteomics (SCP), it does have several limitations that should be acknowledged. These limitations are primarily due to necessary simplifications made during the design of the simulation and the lack of existing datasets with suitable complexity to inform more realistic modelling.

Simplified data generation. The simulation assumes that protein intensities (after log transformation) follow a normal distribution. While this assumption simplifies the mathematical and computational treatment of the data, it may not fully reflect the actual distribution of SCP data (cf. protein "NACAM" in Figure 16 A), which could exhibit heteroskedasticity, heavy tails, or nonlinear behaviour. Similarly, noise is modelled normally distributed across all conditions, ignoring the possibility of variance structures that depend on biological state.

Arbitrary effect modeling. The effects introduced to simulate patient, cell type, and treatment variability are not empirically derived but instead generated using arbitrary parameters based on scaled normal distributions. While this approach allows control over effect magnitudes and facilitates benchmarking, it does not guarantee that the simulated effects match those observed in real SCP datasets. In particular, we scale all inserted effects proportionally to the protein’s mean log-intensity, which assumes that relative effect sizes are constant across the intensity spectrum, an assumption that may not hold in practice.

No interaction effects. An important simplification of our framework is the lack of interaction terms between design factors. For instance, in real biological systems, patients may exhibit variable responses to treatment, and different cell types may respond heterogeneously to the same perturbation. Modelling these interactions would more accurately reflect the strengths of single-cell data, which enables fine-grained characterization of subpopulation-specific responses. However, for this study, we aimed to establish a baseline comparison between pseudobulk and standard SCP analysis pipelines. Introducing complex interactions would have increased the dimensionality and interpretation complexity of the simulation without necessarily improving our ability to isolate the core differences between modelling approaches.

Simplistic missingness insertion. In our simulation, missing values are introduced at random (MAR), with each protein assigned a fixed missingness probability derived from empirical data. In real datasets, missingness is often not at random (MNAR) and can be condition-dependent, for example, differentially abundant proteins may be more or less likely to be missing in one condition versus another. Such patterns are not captured here, and this could affect the realism of the benchmark, particularly for methods that are sensitive to missing data mechanisms.

Balanced design assumption. Our simulated datasets are constructed with balanced designs: all combinations of patients and cell types have the same number of cells. This simplifies the analysis, but does not reflect the reality of single-cell experiments, where rare cell types or uneven capture efficiencies often lead to substantial imbalance. In real scenarios, such imbalance could affect the reliability of aggregation-based methods and the robustness of differential analysis.

Only one dataset used. Finally, our simulation is parameterised based on a single dataset (Brunner et al., 2022), chosen for its label-free design. However, SCP datasets can vary significantly depending on the experimental protocol, instrumentation, and sample type. As such, the conclusions drawn from our benchmark may not generalize directly to datasets with different noise levels, batch effects, or missingness structures.

In summary, while our simulation framework is a useful starting point for assessing pseudobulk strategies in SCP, its simplifying assumptions regarding distributions, effect structures, missingness, and balance limit its biological realism. Future simulation studies or empirical validations on more complex real-world datasets will be essential to further confirm the findings and refine methodological recommendations.

2.3 Towards More Realistic Simulations

To further evaluate the performance of pseudobulk approaches in single-cell proteomics (SCP), future work should be grounded in simulations based on real datasets that already encapsulate both patient and cell type effects. Ideally, such datasets would allow us to model realistic biological and technical variability. Instead of simulating all layers of variation from scratch, one could simply inject a known treatment effect into the dataset and benchmark different modeling strategies based on their ability to detect this signal. This would produce more realistic and generalizable evaluations of method performance.

Unfortunately, SCP datasets that provide this level of design complexity, multiple patients, annotated cell types, and sufficient depth, are currently not publicly available. However, a promising dataset is currently being acquired. This dataset will represent a major step forward in the field: it will include over 50,000 single cells, derived from more than 4 patients and at least 8 distinct cell types.

During the course of this work, we obtained early access to a subset of this dataset, comprising over 10,000 cells. We began testing simulation strategies using this partial dataset. However, this attempt was unsuccessful. The data exploration showed strong batch effects. In addition, we found that the data we had access to already underwent preprocessing. This preprocessing step, which was conducted before we received the data, appears to have removed part of the patient effect, making it impossible to isolate and model. As a result, we were unable to construct reliable simulations based on this subset.

Nonetheless, we anticipate gaining access to the unprocessed and complete dataset in the near future. This will open the door to much more realistic simulation strategies. With it, we aim to:

- Preserve inherent patient and cell type variation.
- Insert controlled treatment effects using various strategies:
 - With or without interaction with cell type.
 - Scaled or unscaled with respect to protein intensity.
- Evaluate method robustness across real-world noise profiles and unbalanced cell distributions.
- Test other methods (*Trigler*, *limpa*, ...)

This upcoming dataset will allow us to test pseudobulk and alternative methods under conditions that are both controlled and biologically realistic, reducing the gap between synthetic simulation and real data application.

Conclusions

In this thesis, we addressed two major aspects of computational workflows in single-cell proteomics: performance optimisation of core software infrastructure and the evaluation of a new SCP modelling strategy.

We began by conducting a systematic performance analysis of the existing `QFeatures` and `scp` packages. This diagnostic step was essential to identify the main computational bottlenecks in real-world workflows. Through detailed profiling of runtime and memory usage, we pinpointed specific functions, most notably `aggregateFeatures` and `subsetByColData`, as the most performance-critical components of the infrastructure. These insights directly guided our optimisation efforts and ensured that development resources were focused where they would yield the greatest benefit.

Building on these findings, we implemented a series of optimisations. These changes led to a substantial 96% reduction in total runtime and approximately 70% reduction in memory usage. We also outlined future directions for improving scalability further, including the use of parallelisation (e.g., via `BiocParallel`) and more efficient data representations such as NA-aware sparse arrays and on-disk storage formats. While powerful, these approaches require careful balancing of trade-offs between speed, memory usage, implementation complexity and maintainability.

In parallel, we used a simulation-based framework to evaluate pseudobulk modelling strategies for SCP data. Our results show that simple aggregation methods such as the mean or median perform comparably to advanced tools like `msqrob2`, while offering improved computational efficiency. Moreover, model performance remained robust even at low cell counts, which is encouraging for studies with limited sampling throughput. However, some aggregation strategies, such as `sum` due to its poor performance and `robustSummary` due to its excessive computational cost, were found to be suboptimal.

We also discussed the limitations of our simulation framework, including its simplified assumptions, the absence of interaction effects, and the use of a single dataset for parameter estimation. These limitations point to the need for more biologically realistic datasets to better benchmark SCP analysis methods under complex, real-world conditions.

Encouragingly, a new large-scale SCP dataset is currently being acquired, comprising over 50,000 cells, multiple patients, and a rich diversity of cell types. This resource will enable the development of more realistic simulations and provide a robust foundation for benchmarking advanced statistical models at scale.

In summary, this work contributes to both immediate software improvements to SCP analysis pipelines in R and a roadmap for scalable, efficient data modelling strategies. By starting with a careful assessment of current infrastructure and then applying focused, impactful optimisations, we have enhanced the performance and usability of `QFeatures` and `scp`. These advances prepare the way for scalable analysis of increasingly large and complex SCP datasets. These advances are important, as future datasets will inevitably grow in size, both in the number of cells profiled and in the inclusion of multiple patient replicates. Such larger datasets are essential to address biological questions with sufficient statistical power. They enable robust detection of treatment effects (or other effect of interest) while disentangling them from unwanted biological or technical variability [59]. In particular, patient replication in SCP will allow separation of within-patient heterogeneity from between-patient variation, an important step toward drawing reliable biomedical conclusions. Pseudobulking, as explored here, offers a promising solution for handling such data efficiently, although further work will be needed to test its application as more appropriate datasets become available. Finally, this study underscores the critical role of research software engineers with combined expertise in biology and software engineering, whose contributions are vital

to developing and sustaining the high-quality, scalable computational tools that modern medical research increasingly demands.

Acknowledgments

I would first like to express my deepest gratitude to my co-supervisor, Laurent Gatto, for his unwavering daily support and the considerable time and effort he invested in guiding this work. His commitment and expertise have been invaluable throughout the course of my thesis. I am equally grateful to my second co-supervisor, Matthieu Defrance, whose insightful perspective from outside the immediate field provided fresh and constructive viewpoints that greatly enriched this research.

I would also like to warmly thank Christophe Vanderaa, developer of *scp*, whose valuable and timely advice provided key insights that significantly advanced my work.

I extend my thanks to all members of the CBIO lab: Anaghim, Axelle, Enes, Jérôme, Julie, Guillaume, Loïc, Matthieu, Philippe, Riya, Samuel, and Shabnam, for making this experience particularly enjoyable. Their diverse backgrounds and expertise have offered me a wide range of perspectives, which proved highly beneficial in shaping my work.

I am also deeply appreciative of Corinne, Jean-Luc, and Elfie, whose steadfast support and readiness to listen to my progress have been a constant source of encouragement.

Finally, I would like to sincerely thank the members of the jury for dedicating their time and effort to evaluate my work.

Code Availability

All scripts that were used in this project can be found in the following GitHub repository: <https://github.com/UCLouvain-CBIO/2024-mth-leopold-guyot>

Author's Note

Portions of the text in this thesis were revised for grammar, spelling, and syntax using AI-assisted writing tools and grammar-checking software. These tools were employed exclusively to improve clarity and readability; all scientific content, interpretations, and conclusions are solely the responsibility of the author.

Bibliography

- [1] Patrick Stelmach and Andreas Trumpp. Leukemic stem cells and therapy resistance in acute myeloid leukemia. *Haematologica*, 108(2):353–366, February 2023.
- [2] Alexander Wacławiczek, Aino-Maija Leppä, Simon Renders, and Andreas Trumpp. An arms-race against resistance: leukemic stem cells and lineage plasticity. *Molecular Oncology*, 18(3):475–478, 2024. [_eprint: https://febs.onlinelibrary.wiley.com/doi/pdf/10.1002/1878-0261.13606](https://febs.onlinelibrary.wiley.com/doi/pdf/10.1002/1878-0261.13606).
- [3] Bogdan Budnik, Ezra Levy, Guillaume Harmange, and Nikolai Slavov. SCoPE-MS: mass spectrometry of single mammalian cells quantifies proteome heterogeneity during cell differentiation. *Genome Biology*, 19(1):161, October 2018.
- [4] Alexander Franks, Edoardo Airoidi, and Nikolai Slavov. Post-transcriptional regulation across human tissues. *PLOS Computational Biology*, 13(5):e1005535, May 2017. Publisher: Public Library of Science.
- [5] Wei Wei, Young Shik Shin, Min Xue, Tomoo Matsutani, Kenta Masui, Huijun Yang, Shiro Ikegami, Yuchao Gu, Ken Herrmann, Dazy Johnson, Xiangming Ding, Kiwook Hwang, Jungwoo Kim, Jian Zhou, Yapeng Su, Xinmin Li, Bruno Bonetti, Rajesh Chopra, C. David James, Webster K. Cavenee, Timothy F. Cloughesy, Paul S. Mischel, James R. Heath, and Beatrice Gini. Single-Cell Phosphoproteomics Resolves Adaptive Signaling Dynamics and Informs Targeted Combination Therapy in Glioblastoma. *Cancer Cell*, 29(4):563–573, April 2016. Publisher: Elsevier.
- [6] Laura L. Baers, Lisa M. Breckels, Lauren A. Mills, Laurent Gatto, Michael J. Deery, Tim J. Stevens, Christopher J. Howe, Kathryn S. Lilley, and David J. Lea-Smith. Proteome Mapping of a Cyanobacterium Reveals Distinct Compartment Organization and Cell-Dispersed Metabolism1 [CC-BY]. *Plant Physiology*, 181(4):1721–1738, December 2019.
- [7] Olivia J. Conway, Josie A. Christopher, Lisa M. Breckels, Hanqi Li, Dilip Menon, Lu Liang, Satish Patel, David C. Gershlick, David B. Savage, and Michael P. Weekes. Dynamic subcellular proteomics identifies novel regulators of adipocyte insulin action. *bioRxiv*, pages 2025–03, 2025. Publisher: Cold Spring Harbor Laboratory.
- [8] Ezra Levy and Nikolai Slavov. Single cell protein analysis for systems biology. *Essays in biochemistry*, 62(4):595–605, October 2018.
- [9] Lauren J. Tracey, Yeji An, and Monica J. Justice. CyTOF: An Emerging Technology for Single-Cell Proteomics in the Mouse. *Current Protocols*, 1(4):e118, 2021. [_eprint: https://currentprotocols.onlinelibrary.wiley.com/doi/pdf/10.1002/cpz1.118](https://currentprotocols.onlinelibrary.wiley.com/doi/pdf/10.1002/cpz1.118).
- [10] Marlon Stoeckius, Christoph Hafemeister, William Stephenson, Brian Houck-Loomis, Pratip K. Chattopadhyay, Harold Swerdlow, Rahul Satija, and Peter Smibert. Simultaneous epitope and transcriptome measurement in single cells. *Nature Methods*, 14(9):865–868, September 2017. Publisher: Nature Publishing Group.
- [11] Jia-Ren Lin, Mohammad Fallahi-Sichani, Jia-Yun Chen, and Peter K. Sorger. Cyclic Immunofluorescence (CyclIF), A Highly Multiplexed Method for Single-cell Imaging. *Current Protocols in Chemical Biology*, 8(4):251–264, 2016. [_eprint: https://currentprotocols.onlinelibrary.wiley.com/doi/pdf/10.1002/cpch.14](https://currentprotocols.onlinelibrary.wiley.com/doi/pdf/10.1002/cpch.14).
- [12] John W. Hickey, Yuqi Tan, Garry P. Nolan, and Yury Goltsev. Strategies for Accurate Cell Type Identification in CODEX Multiplexed Imaging Data. *Frontiers in Immunology*, 12, August 2021. Publisher: Frontiers.
- [13] Indranil Paul, Carl White, Isabella Turcinovic, and Andrew Emili. Imaging the future: the emerging era of single-cell spatial proteomics. *The FEBS Journal*, 288(24):6990–7001, 2021. [_eprint: https://febs.onlinelibrary.wiley.com/doi/pdf/10.1111/febs.15685](https://febs.onlinelibrary.wiley.com/doi/pdf/10.1111/febs.15685).
- [14] Michael A. Innis, David H. Gelfand, John J. Sninsky, and Thomas J. White. *PCR Protocols: A Guide to Methods and Applications*. Academic Press, December 2012. Google-Books-ID: Z5jwZ2rbVe8C.
- [15] Andreas-David Brunner, Marvin Thielert, Catherine Vasilopoulou, Constantin Ammar, Fabian Coscia, Andreas Mund, Ole B Hoerning, Nicolai Bache, Amalia Apalategui, Markus Lubeck, Sabrina Richter, David S Fischer, Oliver Raether, Melvin A Park, Florian Meier, Fabian J Theis, and Matthias Mann. Ultra-high sensitivity mass spectrometry quantifies single-cell proteome changes upon perturbation. *Molecular Systems Biology*, 18(3):e10798, March 2022. Publisher: John Wiley & Sons, Ltd.
- [16] Andrew Leduc, R. Gray Huffman, Joshua Cantlon, Saad Khan, and Nikolai Slavov. Exploring functional protein covariation across single cells using nPOP. *Genome Biology*, 23(1):261, December 2022.
- [17] Harrison Specht, Edward Emmott, Aleksandra A. Petelski, R. Gray Huffman, David H. Perlman, Marco Serra, Peter Kharchenko, Antonius Koller, and Nikolai Slavov. Single-cell proteomic and transcriptomic analysis of macrophage heterogeneity using SCoPE2. *Genome Biology*, 22(1):50, January 2021.
- [18] Ryan T. Kelly. Single-cell Proteomics: Progress and Prospects. *Molecular & Cellular Proteomics*, 19(11):1739–1748, November 2020.

- [19] Nikolai Slavov. Single-cell protein analysis by mass spectrometry. *Current Opinion in Chemical Biology*, 60:1–9, February 2021.
- [20] Yong Chiang Tan, Teck Yew Low, Pey Yee Lee, and Lay Cheng Lim. Single-cell proteomics by mass spectrometry: Advances and implications in cancer research. *PROTEOMICS*, 24(12-13):2300210, 2024. [_eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/pmic.202300210](https://onlinelibrary.wiley.com/doi/pdf/10.1002/pmic.202300210).
- [21] Hayley M. Bennett, William Stephenson, Christopher M. Rose, and Spyros Darmanis. Single-cell proteomics enabled by next-generation sequencing or mass spectrometry. *Nature Methods*, 20(3):363–374, March 2023. Publisher: Nature Publishing Group.
- [22] Siqi Li, Shuwei Li, Siqi Liu, and Yan Ren. MS-based Solutions for Single Cell Proteomics. *Genomics, Proteomics & Bioinformatics*, page qzaf012, February 2025.
- [23] Helena L. Crowell, Charlotte Soneson, Pierre-Luc Germain, Daniela Calini, Ludovic Collin, Catarina Raposo, Dheeraj Malhotra, and Mark D. Robinson. muscat detects subpopulation-specific state transitions from multi-sample multi-condition single-cell transcriptomics data. *Nature Communications*, 11(1):6077, November 2020. Publisher: Nature Publishing Group.
- [24] W. A. Bonner, H. R. Hulett, R. G. Sweet, and L. A. Herzenberg. Fluorescence Activated Cell Sorting. *Review of Scientific Instruments*, 43(3):404–409, March 1972.
- [25] Ying Zhu, Paul D. Piehowski, Rui Zhao, Jing Chen, Yufeng Shen, Ronald J. Moore, Anil K. Shukla, Vladislav A. Petyuk, Martha Campbell-Thompson, Clayton E. Mathews, Richard D. Smith, Wei-Jun Qian, and Ryan T. Kelly. Nanodroplet processing platform for deep and quantitative proteome profiling of 10–100 mammalian cells. *Nature Communications*, 9(1):882, February 2018. Publisher: Nature Publishing Group.
- [26] Jason Derks, Kevin McDonnell, Nathan Wamsley, Peyton Stewart, Maddy Yeh, Harrison Specht, and Nikolai Slavov. Increasing mass spectrometry throughput using time-encoded sample multiplexing, May 2025. Pages: 2025.05.22.655515 Section: New Results.
- [27] Jason Derks, Andrew Leduc, Georg Wallmann, R. Gray Huffman, Matthew Willetts, Saad Khan, Harrison Specht, Markus Ralser, Vadim Demichev, and Nikolai Slavov. Increasing the throughput of sensitive proteomics by plexDIA. *Nature Biotechnology*, 41(1):50–59, January 2023. Publisher: Nature Publishing Group.
- [28] Zhen Long, Zixi Zhao, Xingliang Fan, and Xi Luo. Comparison of analytical-flow, micro-flow and nano-flow LC-MS/MS for sub-proteome analysis. *Journal of Pharmaceutical and Biomedical Analysis*, 252:116484, January 2025.
- [29] Zi-Yi Li, Min Huang, Xiu-Kun Wang, Ying Zhu, Jin-Song Li, Catherine C. L. Wong, and Qun Fang. Nanoliter-Scale Oil-Air-Droplet Chip-Based Single Cell Proteomic Analysis. *Analytical Chemistry*, 90(8):5430–5438, April 2018. Publisher: American Chemical Society.
- [30] Chia-Feng Tsai, Rui Zhao, Sarah M. Williams, Ronald J. Moore, Kendall Schultz, William B. Chrisler, Ljiljana Pasatolic, Karin D. Rodland, Richard D. Smith, Tujin Shi, Ying Zhu, and Tao Liu. An Improved Boosting to Amplify Signal with Isobaric Labeling (iBASIL) Strategy for Precise Quantitative Single-cell Proteomics *. *Molecular & Cellular Proteomics*, 19(5):828–838, May 2020. Publisher: Elsevier.
- [31] Aleksandra A. Petelski, Edward Emmott, Andrew Leduc, R. Gray Huffman, Harrison Specht, David H. Perlman, and Nikolai Slavov. Multiplexed single-cell proteomics using SCoPE2. *Nature Protocols*, 16(12):5398–5425, December 2021. Publisher: Nature Publishing Group.
- [32] Marvin Thielert, Ericka CM Itang, Constantin Ammar, Florian A Rosenberger, Isabell Bludau, Lisa Schweizer, Thierry M Nordmann, Patricia Skowronek, Maria Wahle, Wen-Feng Zeng, Xie-Xuan Zhou, Andreas-David Brunner, Sabrina Richter, Mitchell P Levesque, Fabian J Theis, Martin Steger, and Matthias Mann. Robust dimethyl-based multiplex-DIA doubles single-cell proteome depth via a reference channel. *Molecular Systems Biology*, 19(9):e11503, September 2023. Publisher: John Wiley & Sons, Ltd.
- [33] Victoria Stodden, Peixuan Guo, and Zhaokun Ma. Toward Reproducible Computational Research: An Empirical Analysis of Data and Code Policy Adoption by Journals. *PLOS ONE*, 8(6):e67111, June 2013. Publisher: Public Library of Science.
- [34] Sheeba Samuel and Daniel Mietchen. Computational reproducibility of Jupyter notebooks from biomedical publications. *GigaScience*, 13:giad113, January 2024.
- [35] Awan Afiaz, Andrey Ivanov, John Chamberlin, David Hanauer, Candace Savonen, Mary J. Goldman, Martin Morgan, Michael Reich, Alexander Getka, Aaron Holmes, Sarthak Pati, Dan Knight, Paul C. Boutros, Spyridon Bakas, J. Gregory Caporaso, Guilherme Del Fiol, Harry Hochheiser, Brian Haas, Patrick D. Schloss, James A. Eddy, Jake Albrecht, Andrey Fedorov, Levi Waldron, Ava M. Hoffman, Richard L. Bradshaw, Jeffrey T. Leek, and Carrie Wright. Evaluation of software impact designed for biomedical research: Are we measuring what's meaningful?, June 2023. arXiv:2306.03255 [cs].

- [36] Greg Miller. A Scientist's Nightmare: Software Problem Leads to Five Retractions. *Science*, 314(5807):1856–1857, December 2006. Publisher: American Association for the Advancement of Science.
- [37] Zeeya Merali. Computational science: ...Error. *Nature*, 467(7317):775–777, October 2010. Publisher: Springer Nature.
- [38] Victor R. Basili, Jeffrey C. Carver, Daniela Cruzes, Lorin M. Hochstein, Jeffrey K. Hollingsworth, Forrest Shull, and Marvin V. Zelkowitz. Understanding the High-Performance-Computing Community: A Software Engineer's Perspective. *IEEE Software*, 25(4):29–36, August 2008. Num Pages: 29-36 Place: Los Alamitos, United States Publisher: IEEE Computer Society.
- [39] Stefka Tyanova, Tikira Temu, and Juergen Cox. The MaxQuant computational platform for mass spectrometry-based shotgun proteomics. *Nature Protocols*, 11(12):2301–2319, December 2016. Publisher: Nature Publishing Group.
- [40] Andy T. Kong, Felipe V. Leprevost, Dmitry M. Avtonomov, Dattatreya Mellacheruvu, and Alexey I. Nesvizhskii. MSFragger: fast and comprehensive peptide identification in mass spectrometry-based proteomics. *Nature Methods*, 14(5):513–520, May 2017. Publisher: Nature Publishing Group.
- [41] Michael R. Lazear. Sage: An Open-Source Tool for Fast Proteomics Searching and Quantification at Scale. *Journal of Proteome Research*, 22(11):3652–3659, November 2023. Publisher: American Chemical Society.
- [42] Vadim Demichev, Christoph B. Messner, Spyros I. Vernardis, Kathryn S. Lilley, and Markus Ralser. DIA-NN: neural networks and interference correction enable deep proteome coverage in high throughput. *Nature Methods*, 17(1):41–44, January 2020. Publisher: Nature Publishing Group.
- [43] Shuangshuang Jin, Donald S. Daly, David L. Springer, and John H. Miller. The Effects of Shared Peptides on Protein Quantitation in Label-Free Proteomics by LC/MS/MS. *Journal of Proteome Research*, 7(1):164–169, January 2008. Publisher: American Chemical Society.
- [44] Adriaan Sticker, Ludger Goeminne, Lennart Martens, and Lieven Clement. Robust Summarization and Inference in Proteome-wide Label-free Quantification. *Molecular & Cellular Proteomics*, 19(7):1209–1219, July 2020. Publisher: Elsevier.
- [45] Samuel Grégoire, Christophe Vanderaa, Sébastien Pyr dit Ruys, Christopher Kune, Gabriel Mazzucchelli, Didier Verommen, and Laurent Gatto. Standardized Workflow for Mass-Spectrometry-Based Single-Cell Proteomics Data Processing and Analysis Using the scp Package. In Akos Vegvari, Jaakko Teppo, and Roman A. Zubarev, editors, *Mass Spectrometry Based Single Cell Proteomics*, pages 177–220. Springer US, New York, NY, 2024.
- [46] Christophe Vanderaa and Laurent Gatto. The Current State of Single-Cell Proteomics Data Analysis. *Current Protocols*, 3(1):e658, January 2023.
- [47] Laurent Gatto and Christophe Vanderaa. *QFeatures: Quantitative features for mass spectrometry data*, 2025. R package version 1.19.3, commit aab02fc485d7725d20f5d8455ed9306207b28226.
- [48] Martin Morgan, Valerie Obenchain, Jim Hester, and Hervé Pagès. *SummarizedExperiment: A container (S4 class) for matrix-like assays*, 2025. R package version 1.38.1.
- [49] Robert Amézquita, Aaron Lun, Etienne Becht, Vince Carey, Lindsay Carpp, Ludwig Geistlinger, Federico Marini, Kevin Rue-Albrecht, Davide Risso, Charlotte Soneson, Levi Waldron, Herve Pages, Mike Smith, Wolfgang Huber, Martin Morgan, Raphael Gottardo, and Stephanie Hicks. Orchestrating single-cell analysis with bioconductor. *Nature Methods*, 17:137–145, 2020.
- [50] Adriaan Sticker, Ludger Goeminne, Lennart Martens, and Lieven Clement. Robust summarization and inference in proteome-wide label-free quantification. *Molecular Cellular Proteomics*, 19(7):1209–1219, 2020.
- [51] Christophe Vanderaa and Laurent Gatto. scplainer: using linear models to understand mass spectrometry-based single-cell proteomics data, January 2024. Pages: 2023.12.14.571792 Section: New Results.
- [52] Ludger J. E. Goeminne, Kris Gevaert, and Lieven Clement. Peptide-level Robust Ridge Regression Improves Estimation, Sensitivity, and Specificity in Data-dependent Quantitative Label-free Shotgun Proteomics *. *Molecular & Cellular Proteomics*, 15(2):657–668, February 2016. Publisher: Elsevier.
- [53] Matthew The and Lukas Käll. Integrated Identification and Quantification Error Probabilities for Shotgun Proteomics * [S]. *Molecular & Cellular Proteomics*, 18(3):561–570, March 2019.
- [54] Matthew E. Ritchie, Belinda Phipson, Di Wu, Yifang Hu, Charity W. Law, Wei Shi, and Gordon K. Smyth. limma powers differential expression analyses for RNA-sequencing and microarray studies. *Nucleic Acids Research*, 43(7):e47, April 2015.
- [55] Alan E. Murphy and Nathan G. Skene. A balanced measure shows superior performance of pseudobulk methods in single-cell RNA-sequencing analysis. *Nature Communications*, 13(1):7851, December 2022. Publisher: Nature Publishing Group.

- [56] Sini Junttila, Johannes Smolander, and Laura L. Elo. Benchmarking methods for detecting differential states between conditions from multi-subject single-cell RNA-seq data. *Briefings in Bioinformatics*, 23(5):bbac286, September 2022.
- [57] Sishir Subedi, Tomokazu S. Sumida, and Yongjin P. Park. A highly scalable approach to topic modelling in single-cell data by approximate pseudobulk projection, February 2024. Pages: 2024.02.21.581497 Section: New Results.
- [58] Benjamin Furtwängler, Nil Üresin, Sabrina Richter, Mikkel Bruhn Schuster, Despoina Barmpour, Henrietta Holze, Anne Wenzel, Kirsten Grønbaek, Kim Theilgaard-Mönch, Fabian J. Theis, Erwin M. Schoof, and Bo T. Porse. Mapping the human hematopoietic stem and progenitor cell hierarchy through integrated single-cell proteomics and transcriptomics, July 2024. Pages: 2024.07.05.602277 Section: New Results.
- [59] Alyssa A. Nitz, Jose Humberto Giraldez Chavez, Zachary G. Eliason, and Samuel H. Payne. Are We There Yet? Assessing the Readiness of Single-Cell Proteomics to Answer Biological Hypotheses. *Journal of Proteome Research*, July 2024. Publisher: American Chemical Society.
- [60] Lindsay K. Pino, Brian C. Searle, Eric L. Huang, William Stafford Noble, Andrew N. Hoofnagle, and Michael J. MacCoss. Calibration Using a Single-Point External Reference Material Harmonizes Quantitative Mass Spectrometry Proteomics Data between Platforms and Laboratories. *Analytical Chemistry*, 90(21):13112–13117, November 2018. Publisher: American Chemical Society.
- [61] Christophe Vanderaa and Laurent Gatto. Revisiting the Thorny Issue of Missing Values in Single-Cell Proteomics. *Journal of Proteome Research*, 22(9):2775–2784, September 2023.
- [62] Henry Webel, Yasset Perez-Riverol, Annelaura Bach Nielsen, and Simon Rasmussen. Mass spectrometry-based proteomics data from thousands of HeLa control samples. *Scientific Data*, 11(1):112, January 2024. Publisher: Nature Publishing Group.
- [63] Christophe Vanderaa and Laurent Gatto. Replication of single-cell proteomics data reveals important computational challenges. *Expert Review of Proteomics*, 18(10):835–843, October 2021. Publisher: Taylor & Francis _eprint: <https://doi.org/10.1080/14789450.2021.1988571>.
- [64] Stephen J. Callister, Richard C. Barry, Joshua N. Adkins, Ethan T. Johnson, Wei-jun Qian, Bobbie-Jo M. Webb-Robertson, Richard D. Smith, and Mary S. Lipton. Normalization Approaches for Removing Systematic Biases Associated with Mass Spectrometry and Label-Free Proteomics. *Journal of Proteome Research*, 5(2):277–286, February 2006. Publisher: American Chemical Society.
- [65] Lincoln Harris, William E. Fondrie, Sewoong Oh, and William S. Noble. Evaluating Proteomics Imputation Methods with Improved Criteria. *Journal of Proteome Research*, 22(11):3427–3438, November 2023. Publisher: American Chemical Society.
- [66] Thomas Quinn. *peakRAM: Monitor the Total and Peak RAM Used by an Expression or Function*, 2017. R package version 1.0.2.
- [67] Michel Thiel, Baptiste Féraud, and Bernadette Govaerts. ASCA+ and APCA+: Extensions of ASCA and APCA in the analysis of unbalanced multifactorial designs. *Journal of Chemometrics*, 31(6):e2895, 2017. _eprint: <https://analyticalsciencejournals.onlinelibrary.wiley.com/doi/pdf/10.1002/cem.2895>.
- [68] Hervé Pagès. *SparseArray: High-performance sparse data representation and manipulation in R*, 2025. R package version 1.8.1.
- [69] Sandeep Koranne. Hierarchical Data Format 5 : HDF5. In Sandeep Koranne, editor, *Handbook of Open Source Tools*, pages 191–200. Springer US, Boston, MA, 2011.
- [70] Mike Folk, Gerd Heber, Quincey Koziol, Elena Pourmal, and Dana Robinson. An overview of the HDF5 technology suite and its applications. In *Proceedings of the EDBT/ICDT 2011 Workshop on Array Databases*, AD '11, pages 36–47, New York, NY, USA, March 2011. Association for Computing Machinery.
- [71] Neal Richardson, Ian Cook, Nic Crane, Dewey Dunnington, Romain François, Jonathan Keane, Dragoş Moldovan-Grünfeld, Jeroen Ooms, Jacob Wujciak-Jens, and Apache Arrow. *arrow: Integration to 'Apache' 'Arrow'*, 2025. R package version 21.0.0.
- [72] Donald E. Knuth. Structured Programming with go to Statements. *ACM Comput. Surv.*, 6(4):261–301, December 1974.

Annexes

Annex 1:

	Method	Threshold	Number of cells	Shift value	TPR	FDR
1	msqrob2	0.05	50	0.1	0.24 ± 0.03	0.12 ± 0.01
2	pseudobulkMean	0.05	50	0.1	0.19 ± 0.03	0.09 ± 0.01
3	pseudobulkMedian	0.05	50	0.1	0.20 ± 0.03	0.10 ± 0.00
4	pseudobulkRobustSummary	0.05	50	0.1	0.19 ± 0.03	0.09 ± 0.01
5	pseudobulkSum	0.05	50	0.1	0.08 ± 0.01	0.11 ± 0.04
6	scplainer	0.05	50	0.1	0.96 ± 0.01	0.62 ± 0.01
7	msqrob2	0.05	50	0.15	0.68 ± 0.01	0.08 ± 0.01
8	pseudobulkMean	0.05	50	0.15	0.63 ± 0.01	0.07 ± 0.01
9	pseudobulkMedian	0.05	50	0.15	0.63 ± 0.01	0.07 ± 0.01
10	pseudobulkRobustSummary	0.05	50	0.15	0.63 ± 0.00	0.07 ± 0.01
11	pseudobulkSum	0.05	50	0.15	0.31 ± 0.03	0.08 ± 0.01
12	scplainer	0.05	50	0.15	0.99 ± 0.00	0.61 ± 0.01
13	msqrob2	0.05	50	0.2	0.94 ± 0.01	0.07 ± 0.00
14	pseudobulkMean	0.05	50	0.2	0.92 ± 0.01	0.06 ± 0.01
15	pseudobulkMedian	0.05	50	0.2	0.92 ± 0.01	0.06 ± 0.01
16	pseudobulkRobustSummary	0.05	50	0.2	0.92 ± 0.01	0.06 ± 0.01
17	pseudobulkSum	0.05	50	0.2	0.54 ± 0.01	0.07 ± 0.01
18	scplainer	0.05	50	0.2	1.00 ± 0.00	0.61 ± 0.01
19	msqrob2	0.05	50	0.25	0.99 ± 0.00	0.07 ± 0.00
20	pseudobulkMean	0.05	50	0.25	0.99 ± 0.00	0.06 ± 0.00
21	pseudobulkMedian	0.05	50	0.25	0.99 ± 0.00	0.06 ± 0.01
22	pseudobulkRobustSummary	0.05	50	0.25	0.99 ± 0.00	0.06 ± 0.00
23	pseudobulkSum	0.05	50	0.25	0.71 ± 0.01	0.07 ± 0.01
24	scplainer	0.05	50	0.25	1.00 ± 0.00	0.61 ± 0.00

Table S1. FDR and TPR of models applied on simulated data with different treatment shift values. TPR and FDR columns include the mean and the standard deviation ($\pm$). We here only include the results at an adjusted pvalue of 0.05.

Annex 2:

	Method	Threshold	Number of cells	Shift value	TPR	FDR
1	msqrob2	0.05	10	0.15	0.63 ± 0.03	0.08 ± 0.01
2	pseudobulkMean	0.05	10	0.15	0.60 ± 0.02	0.07 ± 0.01
3	pseudobulkMedian	0.05	10	0.15	0.60 ± 0.02	0.07 ± 0.01
4	pseudobulkRobustSummary	0.05	10	0.15	0.60 ± 0.02	0.07 ± 0.01
5	pseudobulkSum	0.05	10	0.15	0.15 ± 0.03	0.08 ± 0.04
6	scplainer	0.05	10	0.15	0.93 ± 0.01	0.55 ± 0.00
7	msqrob2	0.05	25	0.15	0.67 ± 0.02	0.07 ± 0.02
8	pseudobulkMean	0.05	25	0.15	0.63 ± 0.03	0.06 ± 0.01
9	pseudobulkMedian	0.05	25	0.15	0.63 ± 0.03	0.06 ± 0.02
10	pseudobulkRobustSummary	0.05	25	0.15	0.63 ± 0.02	0.06 ± 0.01
11	pseudobulkSum	0.05	25	0.15	0.23 ± 0.02	0.08 ± 0.01
12	scplainer	0.05	25	0.15	0.98 ± 0.00	0.59 ± 0.01
13	msqrob2	0.05	50	0.15	0.68 ± 0.01	0.08 ± 0.01
14	pseudobulkMean	0.05	50	0.15	0.63 ± 0.01	0.07 ± 0.01
15	pseudobulkMedian	0.05	50	0.15	0.63 ± 0.01	0.07 ± 0.01
16	pseudobulkRobustSummary	0.05	50	0.15	0.63 ± 0.00	0.07 ± 0.01
17	pseudobulkSum	0.05	50	0.15	0.31 ± 0.03	0.08 ± 0.01
18	scplainer	0.05	50	0.15	0.99 ± 0.00	0.61 ± 0.01
19	msqrob2	0.05	100	0.15	0.68 ± 0.01	0.07 ± 0.01
20	pseudobulkMean	0.05	100	0.15	0.64 ± 0.02	0.06 ± 0.01
21	pseudobulkMedian	0.05	100	0.15	0.65 ± 0.02	0.06 ± 0.01
22	pseudobulkRobustSummary	0.05	100	0.15	0.64 ± 0.02	0.05 ± 0.01
23	pseudobulkSum	0.05	100	0.15	0.36 ± 0.02	0.08 ± 0.02
24	scplainer	0.05	100	0.15	1.00 ± 0.00	0.63 ± 0.00

Table S2. FDR and TPR of models applied on different number of simulated cells per condition. TPR and FDR columns include the mean and the standard deviation ($\pm$). We here only include the results at an adjusted pvalue of 0.05.